C922

Tertiary Infotech Academy

Open Source LLM

Fine Tuning Course

COURSE SLIDES · NON-WSQ

Fine Tuning Open Source LLM

Course Code: C922

Conducted by Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Version v1.0 · 2 October 2026

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

COURSE ADMINISTRATION

Welcome & Housekeeping

© 2026 Tertiary Infotech Academy Pte Ltd

About the Trainer



Your Trainer

General Trainer template —
to be completed by the trainer

NAME

TITLE / DESIGNATION

QUALIFICATIONS

AREAS OF EXPERTISE

TRAINING & INDUSTRY EXPERIENCE

CONTACT

© 2026 Tertiary Infotech Academy Pte Ltd

YOUR TRAINER

About the Trainer



Dr. Alfred Ang

Principal Trainer
Tertiary Infotech Academy Pte Ltd

ROLE
Principal Trainer, Tertiary Infotech Academy Pte Ltd

EXPERTISE
Generative AI, large language models, model fine tuning and applied machine learning.

DELIVERS
Courses on generative AI, LLM application development, data science and MLOps.

QUALIFICATIONS
PhD; ACTA / DACE certified; 20+ years in AI and digital transformation.

FOUNDER
Founder and lead instructor at Tertiary Infotech / Tertiary Courses.

© 2026 Tertiary Infotech Academy Pte Ltd

ICE-BREAKER

Let's Know Each Other

1

Who you are

Your name, your organisation and your role.

2

Your Python background

Your experience with Python, PyTorch and the Hugging Face libraries, if any.

3

Your AI background

Have you used an LLM API, or trained or fine-tuned a model of your own before?

4

What you want to build

What would you like to fine-tune a model to do after this course?

© 2026 Tertiary Infotech Academy Pte Ltd

HOUSEKEEPING

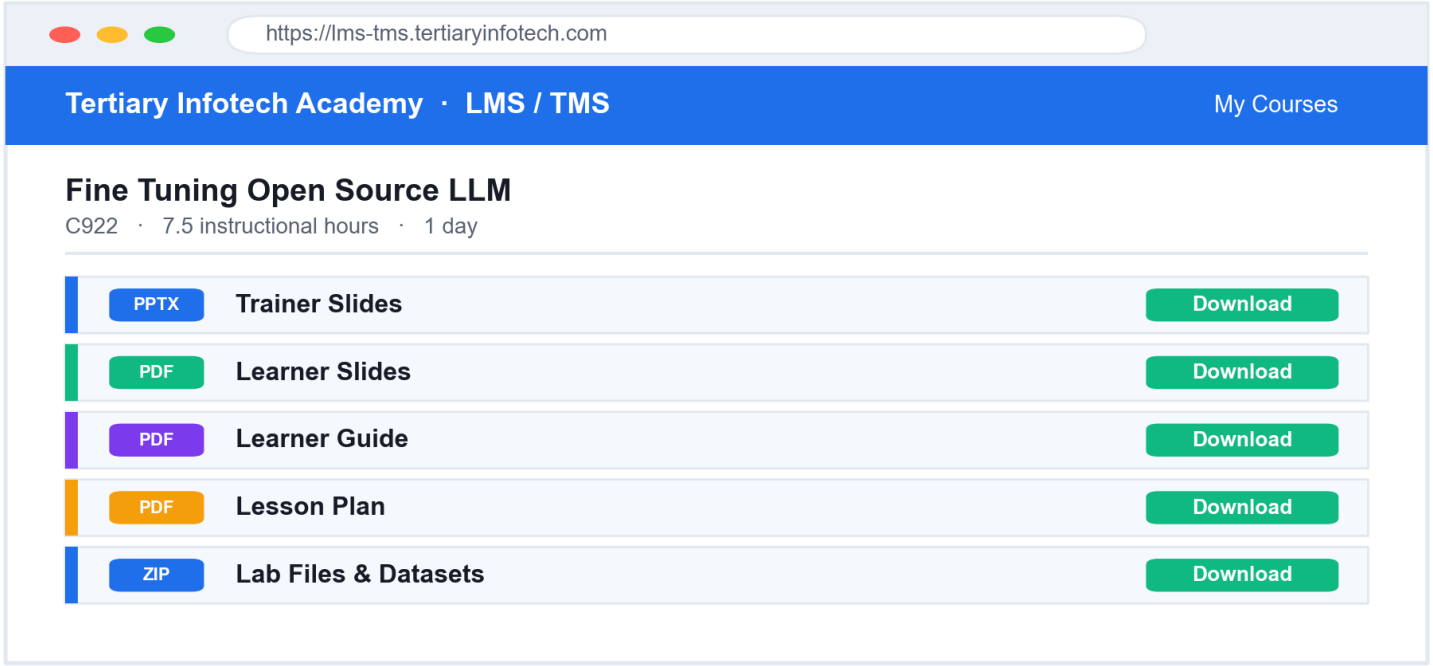
Ground Rules

1. Set your mobile phone to silent mode.
2. Participate actively — no question is too small.
3. Mutual respect: agree to disagree.
4. One conversation at a time.
5. Be punctual; return from breaks on time.
6. Full attendance — stay for the whole day to get the most from the course.

© 2026 Tertiary Infotech Academy Pte Ltd

LMS / TMS · lms-tms.tertiaryinfotech.com

Download Your Course Material



Log in with the e-mail you registered with, open this course, and download all five items before the class starts.

Log in with your registration e-mail, open this course, and download all five items.

© 2026 Tertiary Infotech Academy Pte Ltd

WHAT WE COVER

Course Map — Four Learning Units, One Day

1 day · 7.5 instructional hours · 26 hands-on labs

LU1 · 25%	LU2 · 25%	LU3 · 30%	LU4 · 20%
Data Engineering	**Exploratory Data Analysis**	**Modelling**	**Machine Learning Implementation and Operations**
LO1	LO2	LO3	LO4
+ Labs 2-4, 6 self-paced	+ Labs 8-12 self-paced	+ Labs 13, 14, 16, 18-20 self-paced	+ Labs 22-26 self-paced
· Data repositories · Dataset ingestion · Data transformation	· Clean and sanitize · Feature engineering · Analyse and visualise	· Frame the problem · Select the method · LoRA / QLoRA / Unsloth · Hyperparameter tuning · Evaluate the model	· Benchmark vs SOTA · Bias and fairness · Explainability · Deploy and operate
Labs 1, 5 in class	Lab 7 in class	Labs 15, 17 in class	Lab 21 in class
Morning	Late morning	Afternoon	Late afternoon

Each Learning Unit is one block of the day: its core labs run in class, the rest are self-paced.

Each Learning Unit is one block of the day: its core labs run in class, the rest are self-paced.

© 2026 Tertiary Infotech Academy Pte Ltd

SCHEDULE

Lesson Plan — 1 Day, 9:30am – 5:30pm

Morning

- 9:30am – 12:25pm
  - Welcome · course orientation · core concepts
  - LU1: Data Engineering — Labs 1 and 5 in class
  - LU2: Exploratory Data Analysis — cleaning and sanitisation
  - Lab 7 in class
  - Lunch · 12:25pm – 12:55pm

Afternoon

- 12:55pm – 5:30pm
  - LU2 continued: feature engineering, analysis and visualisation
  - LU3: Modelling — LoRA, QLoRA and Unsloth · Labs 15 and 17 in class
  - LU4: benchmarking, bias, explainability and deployment · Lab 21 in class
  - The other 20 labs are self-paced extension work
  - Course recap · Q&A · Course Feedback

© 2026 Tertiary Infotech Academy Pte Ltd

FOUR LEARNING UNITS · 26 HANDS-ON LABS

Course Outline

1

LU1 — Data Engineering

Data repositories for fine tuning · dataset ingestion · synthetic data generation · transformation into chat/instruction format

2

LU2 — Exploratory Data Analysis

Cleaning and sanitisation · optimisation strategies for higher-quality outputs · feature engineering · ETL pipelines · analysis and visualisation

3

LU3 — Modelling

Framing the fine-tuning problem · multi-modal embeddings · LoRA and QLoRA · Unsloth · training and monitoring · hyperparameter optimisation · advanced loss metrics

4

LU4 — Machine Learning Implementation and Operations

Benchmarking against state of the art · explainability and interpretability · bias and fairness · continuous pre-training pipelines · deployment

© 2026 Tertiary Infotech Academy Pte Ltd

LAB ENVIRONMENT

Set Up Before We Start

1

Google account

Sign in to Google Colab at https://colab.research.google.com — a free T4 GPU is all you need.

2

Select a GPU runtime

Runtime > Change runtime type > T4 GPU. Every lab assumes an attached GPU.

3

Install the stack

pip install unsloth, then --no-deps trl peft accelerate bitsandbytes.

4

Hugging Face account

Free account at huggingface.co for datasets and for pushing your adapter.

5

Lab files

Download the labs ZIP from the LMS — each lab folder carries its own mock dataset.

6

No local GPU needed

Kaggle (2× T4) or a local NVIDIA GPU with 12GB+ VRAM also work.

© 2026 Tertiary Infotech Academy Pte Ltd

CORE CONCEPTS

Why Fine Tuning, and What LoRA Actually Does

© 2026 Tertiary Infotech Academy Pte Ltd

FOUNDATIONS

What is a Large Language Model?

1

A model of language

An LLM such as Llama, GPT or Claude is trained to understand and generate human language.

2

Why 'large'

Large in training data and in parameter count — from one billion to hundreds of billions.

3

Autoregressive

It predicts the next token, one at a time, conditioned on everything written so far.

4

Pre-trained, then adapted

Pre-training builds general capability; adaptation makes it useful for YOUR task.

© 2026 Tertiary Infotech Academy Pte Ltd

How a Base Model Is Created

1. Collect a web-scale text corpus
2. Self-supervised next-token training
3. Costing millions of GPU-hours
4. Producing a general base model
5. Which you then adapt cheaply

© 2026 Tertiary Infotech Academy Pte Ltd

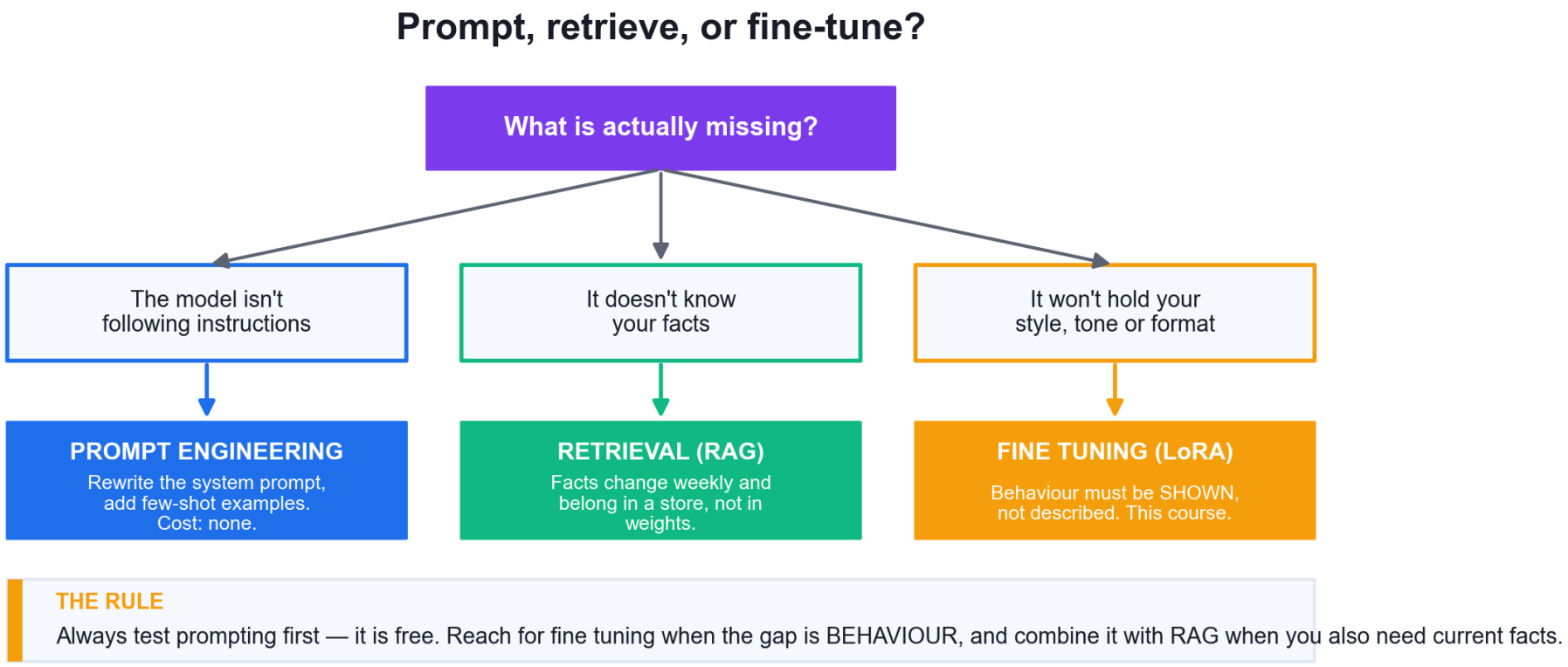
THE ECONOMICS

You will never pre-train. You will always adapt.

Pre-training a base model costs millions of dollars. Adapting one to your domain, style and format costs a few dollars of GPU time — and that gap is what this course is about.

© 2026 Tertiary Infotech Academy Pte Ltd

Prompt, Retrieve, or Fine-Tune?



Choosing wrongly is the most expensive mistake in the field. Diagnose what is MISSING before you spend anything.

© 2026 Tertiary Infotech Academy Pte Ltd

Three Ways to Adapt a Model

Approach	Fixes	Cost	Latency	When to use it
Prompt engineering	Instruction following	None	Worse (long prompt)	Always try this first
Few-shot prompting	Output format	None	Worse (examples in prompt)	A handful of examples suffice
RAG	Missing or changing facts	Low	Worse (retrieval hop)	Knowledge changes weekly
LoRA fine tuning	Style, tone, format, behaviour	Low (1 GPU)	Best (short prompt)	Behaviour must be shown
Full fine tuning	Same as LoRA	Very high	Best	Rarely justified

WHEN IT MATTERS Fine tuning teaches BEHAVIOUR. It does not reliably teach FACTS — facts belong in retrieval, where they can be updated without retraining.

© 2026 Tertiary Infotech Academy Pte Ltd

LO3 · DESIGN DECISION

Should You Fine-Tune This Problem?



YES — prompt or RAG first

If a system prompt and a few examples get you there, stop. It is free, instant, and there is nothing to maintain. Add RAG if the gap is missing facts.

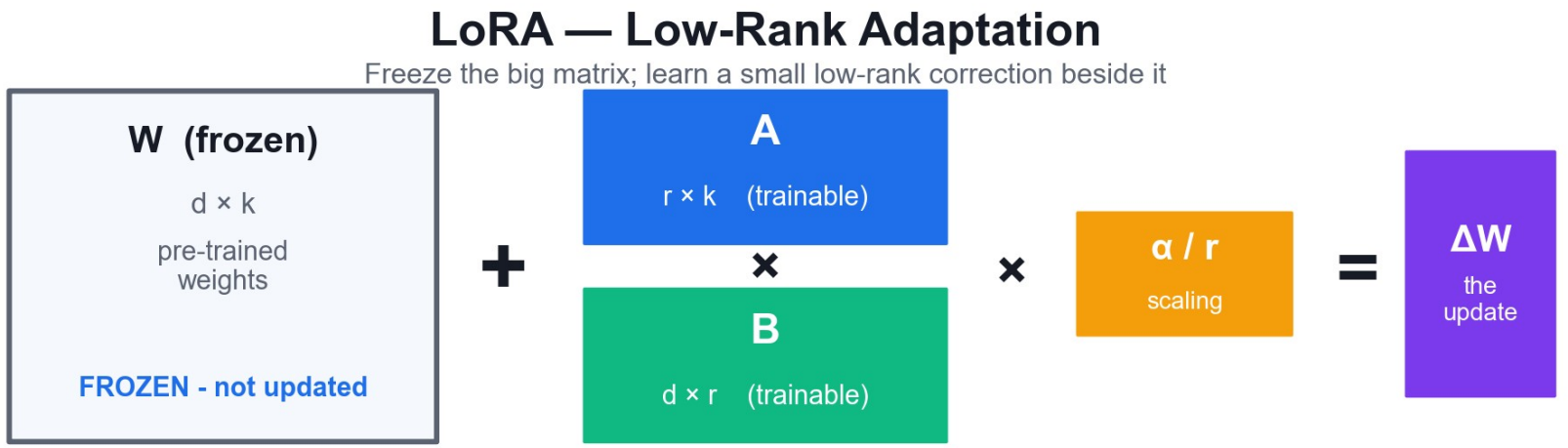
NO — fine-tune it

If the behaviour must be SHOWN through hundreds of examples rather than described, that is exactly what fine tuning is for. This is the case throughout this course.

A useful test: if you cannot write the rule down, but you can show fifty examples of it, you have a fine-tuning problem.

© 2026 Tertiary Infotech Academy Pte Ltd

LoRA — Low-Rank Adaptation



$$h = Wx + (\alpha/r) \cdot BAx$$

B starts at ZERO → BA = 0 → the adapted model begins exactly equal to the base model. Only A and B are trained: r(d + k) parameters instead of d × k.

Freeze W. Learn two small matrices beside it. Because B starts at zero, training begins as an exact no-op and only ever adds.

© 2026 Tertiary Infotech Academy Pte Ltd

LO3 · WHAT YOU ACTUALLY CONFIGURE

The Four LoRA Parameters That Matter

1

r — the rank

Adapter capacity. 8–16 for style and format; 32–64 for genuinely new domain skills.

2

lora_alpha

Update strength through the α/r scaling factor. A common default is alpha = 2r.

3

target_modules

WHERE adaptation happens. Attention-only underfits; ALL linear layers is what works.

4

lora_dropout

Regularisation on the adapter input. 0 is optimised in Unsloth; 0.05 is a safe default.

5

bias

Whether bias terms train. 'none' is standard and is the optimised path.

6

use_rslora

Scale by α/√r instead of α/r, keeping the update stable as the rank rises.

© 2026 Tertiary Infotech Academy Pte Ltd

LO3 · WORKED EXAMPLE

Reading a LoRA Configuration

Every fine-tune in this course is configured by these fields — learn to read them once.

```
from peft import LoraConfig, get_peft_model

config = LoraConfig(
    r              = 16,
    lora_alpha     = 32,      # alpha/r = 2.0
    lora_dropout   = 0.05,
    bias           = 'none',
    task_type      = 'CAUSAL_LM',
    target_modules = [
        'q_proj','k_proj','v_proj','o_proj',
        'gate_proj','up_proj','down_proj',
    ],
)
model = get_peft_model(base_model, config)
```

r = 16

Each adapted layer learns 16·(d_in+d_out) parameters instead of d_in·d_out.

lora_alpha = 32

The update is scaled by α/r = 2.0 before being added to Wx.

target_modules

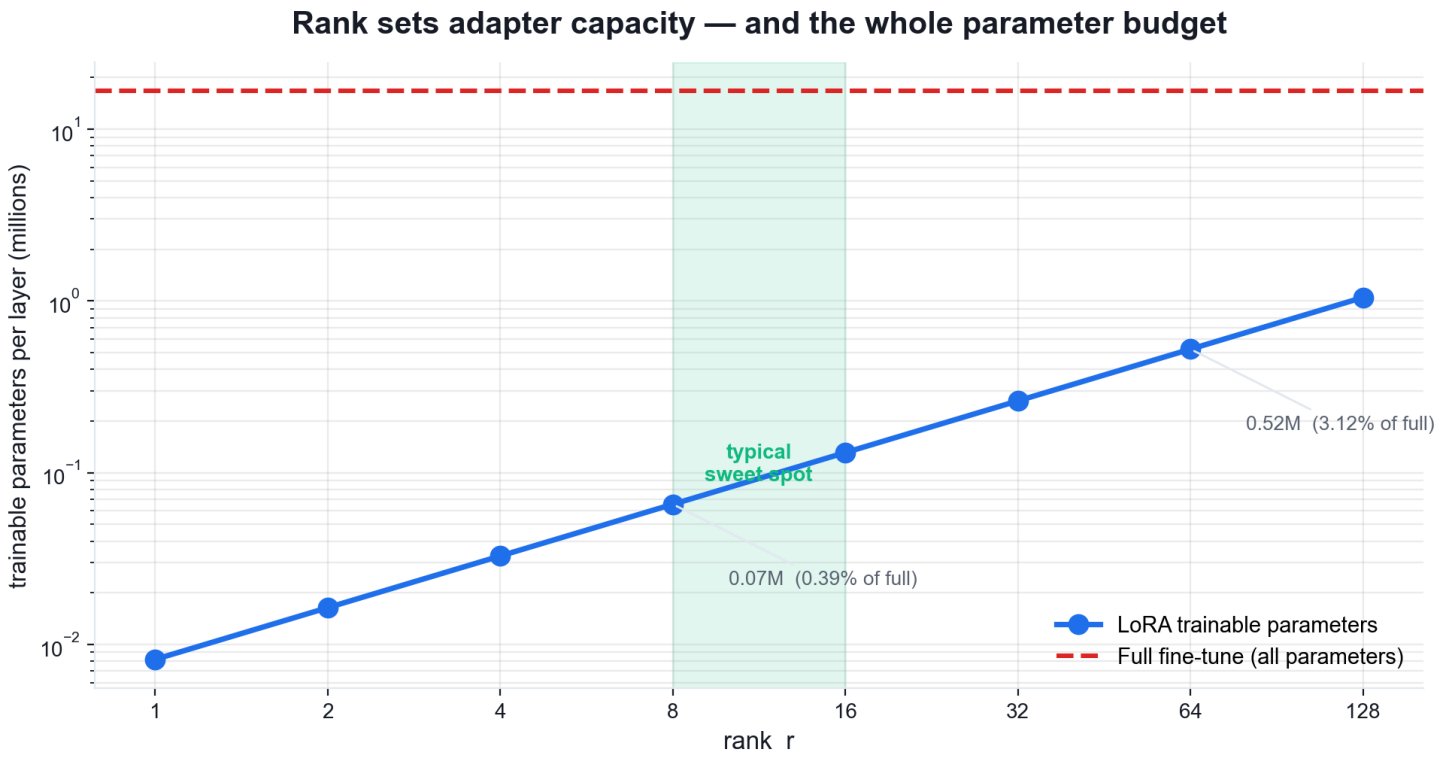
Attention AND the MLP projections. Attention-only is the classic mistake.

task_type

CAUSAL_LM tells PEFT which head and label-shifting convention to use.

© 2026 Tertiary Infotech Academy Pte Ltd

Rank Sets Capacity — and the Whole Budget

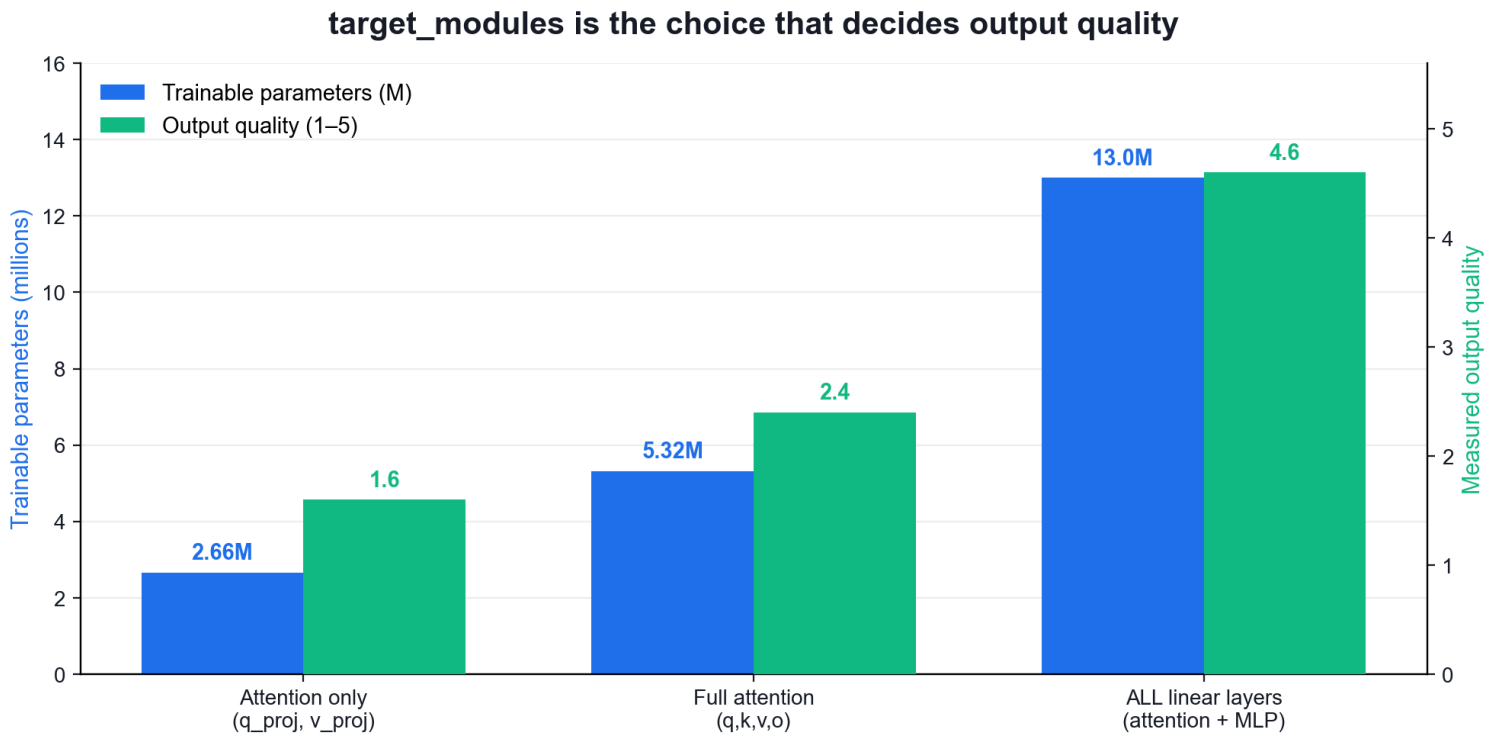


For a 4096×4096 projection. Doubling r doubles capacity and cost, but validation loss usually saturates by r=16.

Capacity rises linearly with r, but validation loss usually saturates by r=16. Raise the rank only when the task needs genuinely new skills.

© 2026 Tertiary Infotech Academy Pte Ltd

target_modules Decides Output Quality



Attention-only targeting underfits badly. Adding the MLP gate/up/down projections roughly doubles the parameters and transforms the output.

Targeting attention only (q_proj, v_proj) underfits badly. Adding the MLP gate/up/down projections is what produces usable output.

© 2026 Tertiary Infotech Academy Pte Ltd

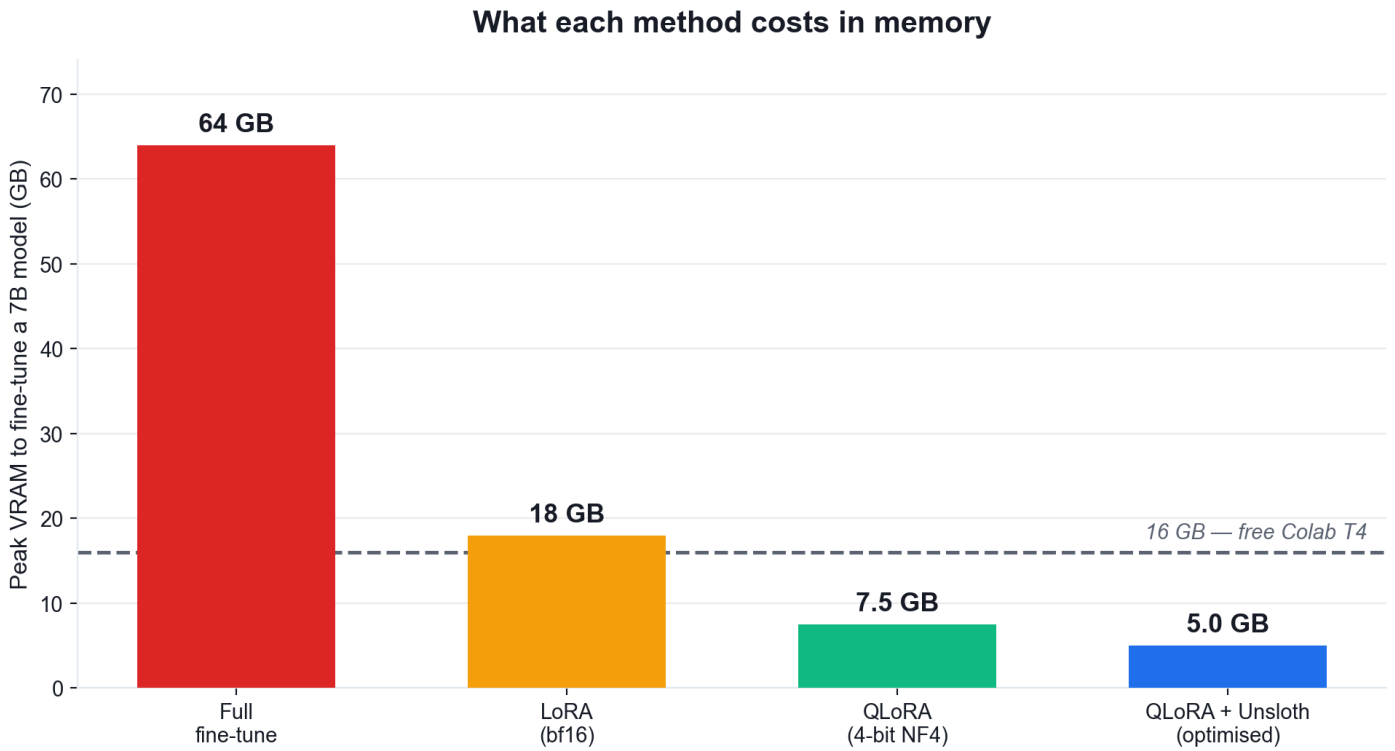
LO3 · QLoRA

QLoRA: quantise the frozen weights, train in full precision.

The base model is stored in 4-bit NF4 and dequantised on the fly for each matmul, while the small adapter stays in bf16. Memory falls three to four times again; quality barely moves.

© 2026 Tertiary Infotech Academy Pte Ltd

What Each Method Costs in Memory



Indicative figures for teaching the trade-off. Only the bottom two fit inside a free Colab session.

VRAM, not FLOPs, is the binding constraint. Only QLoRA fits a 7B fine-tune inside a free Colab session.

© 2026 Tertiary Infotech Academy Pte Ltd

LO3 · THE TRADE-OFF

LoRA vs QLoRA vs Full Fine Tuning

Dimension	Full fine tune	LoRA (bf16)	QLoRA (4-bit)
Trainable params	100%	< 1%	< 1%
Base weights	Updated	Frozen, bf16	Frozen, 4-bit NF4
VRAM (7B model)	~60+ GB	~16–20 GB	~6–8 GB
Quality	Baseline	≈ full	≈ LoRA
Catastrophic forgetting	High risk	Low	Low
Output artifact	Full model (GBs)	Adapter (MBs)	Adapter (MBs)

WHEN IT MATTERS LoRA often BEATS full fine tuning in practice, because freezing the base weights is itself a defence against catastrophic forgetting.

© 2026 Tertiary Infotech Academy Pte Ltd

LO3 · ACCELERATION

What Unsloth Adds

1

Hand-written Triton kernels

Attention, MLP and cross-entropy are re-implemented for the GPU rather than composed from PyTorch ops.

2

Manual backward pass

The gradients are derived by hand instead of being built by autograd, removing redundant work.

3

Roughly 2× faster

Same model, same data, same final loss — just less time on the GPU.

4

70–80% less VRAM

Its own gradient-checkpointing implementation offloads activations intelligently.

5

No accuracy loss

It is a kernel optimisation, not an approximation — the mathematics is unchanged.

6

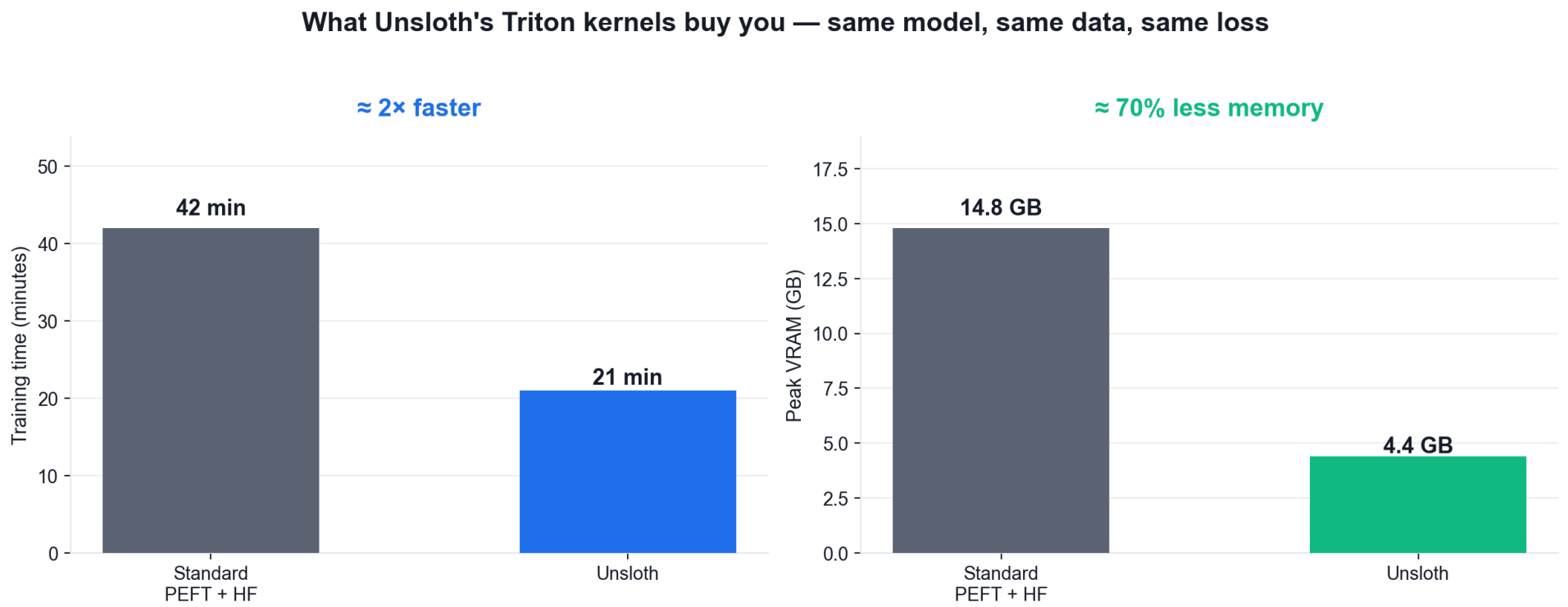
Drop-in API

FastLanguageModel.from_pretrained and get_peft_model mirror the Transformers/PEFT calls.

© 2026 Tertiary Infotech Academy Pte Ltd

LO3 · MEASURED, NOT CLAIMED

What Unsloth's Kernels Buy You



Indicative figures — measure your own in Lab 17 and record them in unsloth_benchmark.csv.

You will measure these numbers yourself in Lab 17 and record them in unsloth_benchmark.csv.

© 2026 Tertiary Infotech Academy Pte Ltd

Where the Memory Actually Goes

Component	Full fine-tune (7B)	LoRA (bf16)	QLoRA (4-bit)
Model weights	14 GB (bf16)	14 GB (frozen)	3.5 GB (NF4)
Gradients	14 GB	~0.05 GB (adapter only)	~0.05 GB
Optimizer state	28 GB (Adam: 2 moments)	~0.1 GB	~0.1 GB (8-bit)
Activations	4-8 GB	2-4 GB (checkpointing)	2-4 GB
Rough total	~60+ GB	~16-20 GB	~6-8 GB

WHEN IT MATTERS The optimizer state is the hidden cost: Adam keeps two moments per trainable parameter. Freezing the base weights removes it almost entirely — that is where LoRA's saving comes from.

© 2026 Tertiary Infotech Academy Pte Ltd

TERMS YOU WILL USE ALL DAY

The Vocabulary of Fine Tuning

1

Adapter

The small trainable module (A and B) added beside a frozen weight matrix.

2

Merging

Folding (α/r)·BA back into W to produce a single standalone model with zero added latency.

3

Quantisation

Storing weights at lower precision (4-bit NF4) and dequantising on the fly for compute.

4

Effective batch size

per_device_batch × gradient_accumulation × devices — what the optimiser actually sees.

5

Gradient checkpointing

Recomputing activations in the backward pass instead of storing them; trades time for VRAM.

6

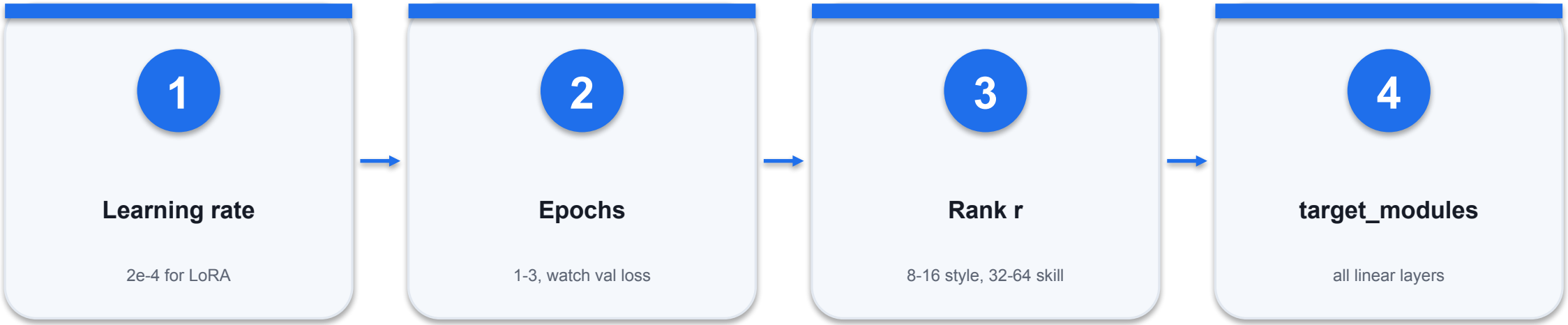
Catastrophic forgetting

Losing general capability while gaining domain capability. Measure it; do not assume it.

© 2026 Tertiary Infotech Academy Pte Ltd

LO3 · IF YOU REMEMBER NOTHING ELSE

The Four Numbers That Decide a Fine-Tune



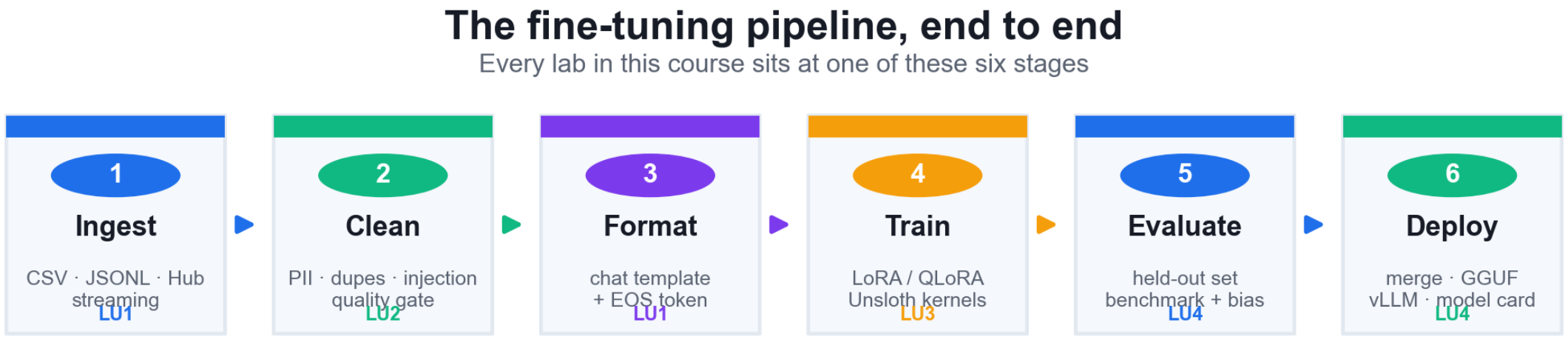
THE ORDER MATTERS

Tune the learning rate first — it dominates every other choice. Only then does it make sense to search over rank, alpha or batch size.

© 2026 Tertiary Infotech Academy Pte Ltd

HOW THE COURSE FITS TOGETHER

The Fine-Tuning Pipeline, End to End



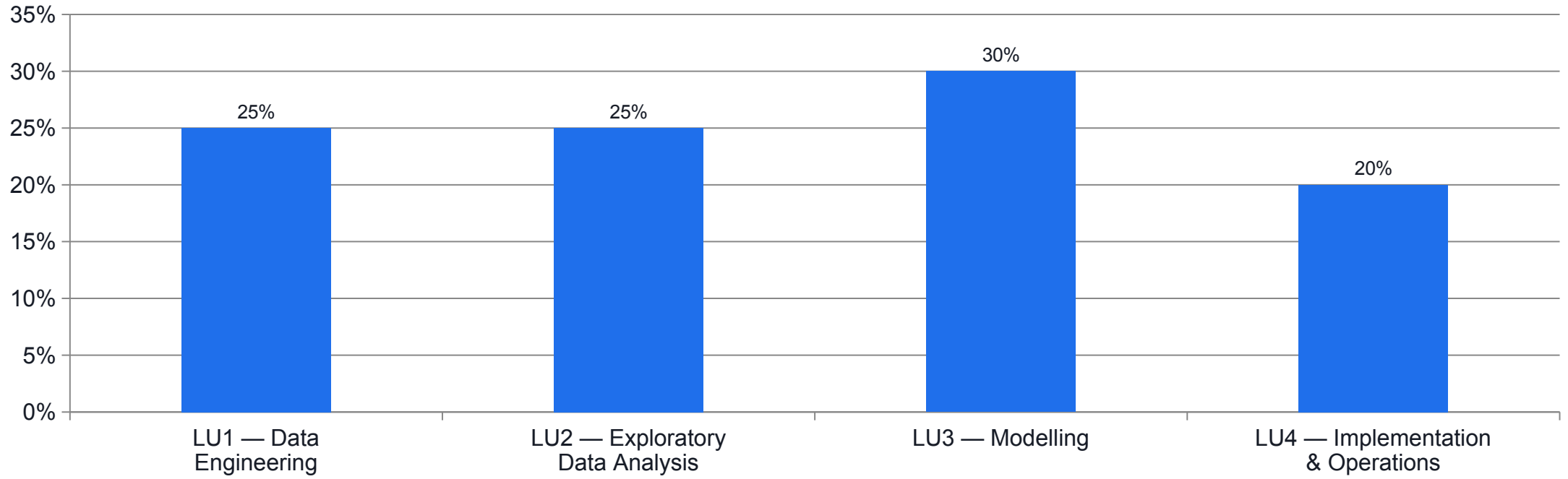
THE FAILURE MODE
Most failed fine-tunes fail at stages 2 and 3 — a dirty dataset or the wrong chat template — not at stage 4. GPU time cannot repair data you did not clean.

Every lab sits at one of these six stages. Most failed fine-tunes fail at stages 2 and 3 — not at stage 4.

© 2026 Tertiary Infotech Academy Pte Ltd

LESSON PLAN · TIME ALLOCATION

Where the Course Time Goes



WHAT THE DATA SHOWS

LU3 carries the largest weighting because LoRA, QLoRA and Unsloth are where the core practical skill sits — expect the most hands-on time there.

© 2026 Tertiary Infotech Academy Pte Ltd

01

LEARNING UNIT 01

Data Engineering

Data repositories for fine tuning · dataset ingestion · synthetic data generation · transformation into chat/instruction format

© 2026 Tertiary Infotech Academy Pte Ltd

LU1 at a Glance

1

Learning outcome

Apply transformation methods to ingest and prepare synthetic data using cloud-based model tools.

2

Hands-on labs

Labs 1 and 5 in class · Labs 2–4 and 6 self-paced

3

Weighting

25% of course time · Morning

4

Topics

Create data repositories for fine tuning · Identify and implement a dataset ingestion solution · Identify and implement a data transformation solution

© 2026 Tertiary Infotech Academy Pte Ltd

Key Concepts — Data Engineering

1. A fine-tuning data repository is versioned, immutable and reproducible — the Hugging Face Hub, an S3/GCS bucket or a local `datasets` cache all serve this role, and every training run must be traceable to an exact dataset revision.

2. Synthetic data generation creates training examples where real data is scarce, sensitive or expensive — self-instruct bootstrapping, teacher-model distillation, template expansion and programmatic augmentation are the four families you will use.

3. Synthetic data must preserve the statistical properties of the real distribution: match the label balance, the length distribution and the domain vocabulary, then de-duplicate against the real set so you do not train on near-copies.

4. Generative architectures differ in what they model: conditional GANs learn a generator/discriminator game, diffusion models learn to reverse a noising process, and autoregressive transformers factorise the sequence probability token by token.

© 2026 Tertiary Infotech Academy Pte Ltd

Key Concepts — Data Engineering (2)

1. Hardware acceleration: GPUs give general-purpose massively-parallel matrix maths with large VRAM; TPUs are systolic-array ASICs tuned for dense matmul at scale. VRAM, not FLOPs, is the binding constraint for fine tuning.

2. Cloud training platforms — Colab, Kaggle, AWS SageMaker, Vertex AI and Azure ML — supply elastic accelerators, managed storage and experiment tracking so a fine-tune is reproducible.

3. The `datasets` library streams, shards, maps and caches datasets far larger than RAM; `load_dataset`, `.map()`, `.filter()` and `.train_test_split()` are the four verbs you need.

4. Transformation converts raw records into the exact chat/instruction template the base model expects — a mismatched template is the single most common cause of a failed fine-tune.

© 2026 Tertiary Infotech Academy Pte Ltd

Key Concepts — Data Engineering (3)

1

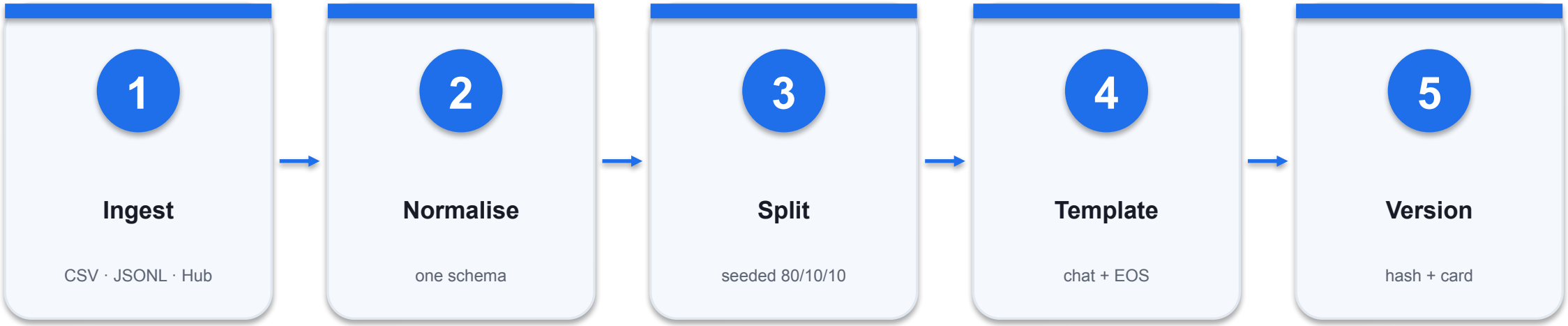
The Alpaca instruction template (instruction / input / response) and the ChatML role template (system / user / assistant) are the two dominant formats; `tokenizer.apply_chat_template()` renders the model's own correct form.

2

Every training example must end with the EOS token, or the fine-tuned model will never learn to stop generating.

© 2026 Tertiary Infotech Academy Pte Ltd

From Raw Records to a Training File



THE POINT

A fine-tuning dataset is not a folder of files — it is a versioned, hashed, reproducible artifact that a training run can be traced back to.

© 2026 Tertiary Infotech Academy Pte Ltd

Four Ways to Make Synthetic Data

Method	How it works	Diversity	Cost	Watch out for
Template expansion	Fill slots in a grammar	Low	Free	Repetitive phrasing
Self-instruct	Teacher model writes new items	High	Inference cost	Teacher's own biases
Back-translation	Translate out and back	Medium	Low	Meaning drift
Programmatic augment	Perturb real examples	Low–medium	Free	Near-duplicates of the seed

WHEN IT MATTERS Whatever the method, prove it statistically: match the length distribution, the label balance and the vocabulary, then de-duplicate against the real set.

© 2026 Tertiary Infotech Academy Pte Ltd

The Chat Template Is Not Optional

The single most common cause of a fine-tune that trains cleanly but behaves badly.

```
# WRONG — a template the base model never saw
text = f'Q: {question}\nA: {answer}'

# RIGHT — the model's OWN template
text = tokenizer.apply_chat_template(
    [{'role':'user',      'content': question},
     {'role':'assistant', 'content': answer}],
    tokenize=False,
)

# ALWAYS append EOS, or it never learns to stop
text = text + tokenizer.eos_token
```

apply_chat_template

Emits the exact special tokens the base model was pre-trained with.

Why it matters

A mismatched template means the model must re-learn the format from scratch.

The EOS token

Without it the model generates until it hits max_new_tokens, every time.

Verify it

Decode with skip_special_tokens=False and LOOK at the tokens.

© 2026 Tertiary Infotech Academy Pte Ltd

Generative Architectures Compared

Family	Generation quality	Training stability	Compute	Typical use
Conditional GAN	Sharp, can mode-collapse	Low — adversarial game	Low	Fast image synthesis
Diffusion	Highest fidelity + diversity	High — simple MSE loss	Very high	Text-to-image, audio
Autoregressive	State of the art for text	High — cross-entropy	High	LLMs, code, multi-modal

WHEN IT MATTERS Diffusion trains stably because its objective is a plain MSE on the added noise — there is no adversarial game to balance.

© 2026 Tertiary Infotech Academy Pte Ltd

Accelerators and Cloud Platforms

1

NVIDIA T4 (16 GB)

Free on Colab. Enough for QLoRA on 1B–8B models — the environment for this course.

2

NVIDIA A100 / H100

40–80 GB. Full fine-tuning up to 7B, or large-scale LoRA training.

3

Google TPU

Systolic-array ASIC, excellent at large dense matmul, but needs XLA-compatible code.

4

AWS SageMaker

Managed training jobs, experiment tracking, S3 storage — production pipelines.

5

Google Vertex AI

Managed training with GPU or TPU, plus model registry and endpoints.

6

Azure ML

Managed compute and MLOps for Microsoft estates.

© 2026 Tertiary Infotech Academy Pte Ltd

WHAT YOU'LL DO

Hands-On Labs — Data Engineering

Labs 1–2

- Set Up the Fine-Tuning Environment on Colab
- Create a Versioned Data Repository for Fine Tuning

Labs 3–4

- Generate Synthetic Training Data
- Ingest Datasets from Multiple Sources

Labs 5–6

- Transform Data into Chat and Instruction Formats
- Compare Generative Architectures and Accelerator Options

© 2026 Tertiary Infotech Academy Pte Ltd

LU1 · HANDS-ON · IN CLASS

Set Up the Fine-Tuning Environment on Colab

LAB 1

Prepare a reproducible GPU environment and verify the accelerator, VRAM and library stack

Open a free Colab GPU runtime, install the Unsloth + PEFT + TRL stack, and verify the GPU, the available VRAM and every library version. You then record the environment in a reproducibility manifest so the same run can be repeated later.

YOU'LL BUILD

A verified GPU notebook environment and an env_manifest.json recording the GPU model, VRAM, CUDA version and the exact version of every library used in the course.

TOOLCHAIN

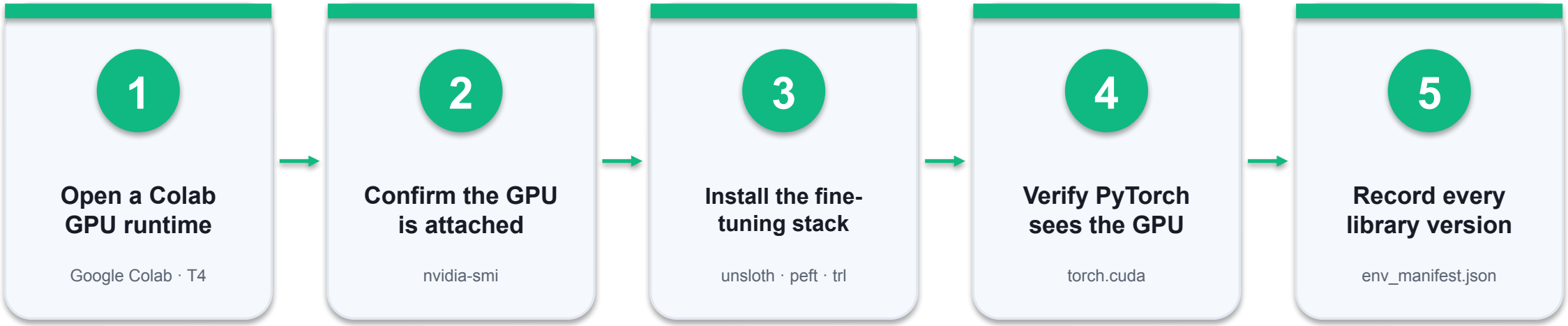
Google Colab, NVIDIA T4, PyTorch, Unsloth, PEFT, TRL, bitsandbytes

DONE WHEN

nvidia-smi shows a T4 (or better), torch.cuda.is_available() is True, env_manifest.json exists with all seven library versions filled in, and the test model loads and unloads without an out-of-memory error.

© 2026 Tertiary Infotech Academy Pte Ltd

How Lab 1 Runs



THE POINT

Prepare a reproducible GPU environment and verify the accelerator, VRAM and library stack

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 1 · VERIFY

Set Up the Fine-Tuning Environment on Colab

✅ Expected result

nvidia-smi shows a T4 (or better), torch.cuda.is_available() is True, env_manifest.json exists with all seven library versions filled in, and the test model loads and unloads without an out-of-memory error.

IF IT DOESN'T WORK

CUDA out of memory

Restart the runtime, lower per_device_train_batch_size, raise gradient_accumulation_steps.

No GPU attached

Runtime > Change runtime type > T4 GPU, then re-run from the first cell.

ModuleNotFoundError

Re-run the pip install cell; on Colab, restart the session after installing unsloth.

© 2026 Tertiary Infotech Academy Pte Ltd

LU1 · HANDS-ON · SELF-PACED

Create a Versioned Data Repository for Fine Tuning

LAB 2

Build an immutable, versioned, reproducible data repository for a fine-tuning corpus

Take a raw customer-support export and turn it into a proper fine-tuning data repository: a content-hashed, versioned, split dataset with a dataset card, so any future training run can be traced back to the exact bytes it was trained on.

YOU'LL BUILD

A versioned dataset repository at data_repo/v1/ containing train/validation/test splits, a manifest.json with a SHA-256 content hash and row counts, and a DATASET_CARD.md.

TOOLCHAIN

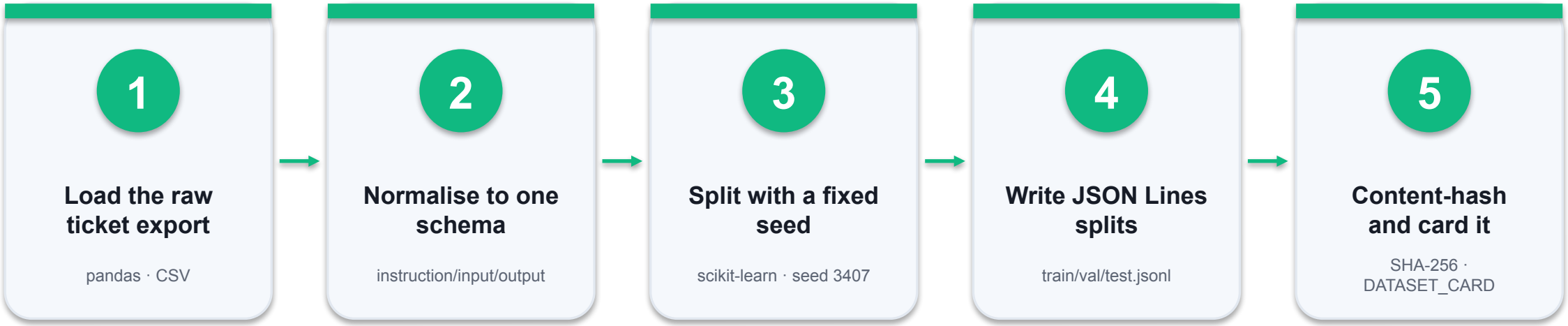
Hugging Face datasets, pandas, hashlib, JSON Lines

DONE WHEN

data_repo/v1/ contains three JSONL splits whose row counts sum to the number of usable records, manifest.json records a SHA-256 for each split, DATASET_CARD.md exists, and load_dataset() returns a DatasetDict with all three splits.

© 2026 Tertiary Infotech Academy Pte Ltd

How Lab 2 Runs



THE POINT

Build an immutable, versioned, reproducible data repository for a fine-tuning corpus

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 2 · VERIFY

Create a Versioned Data Repository for Fine Tuning

✅ Expected result

data_repo/v1/ contains three JSONL splits whose row counts sum to the number of usable records, manifest.json records a SHA-256 for each split, DATASET_CARD.md exists, and load_dataset() returns a DatasetDict with all three splits.

IF IT DOESN'T WORK

CUDA out of memory

Restart the runtime, lower per_device_train_batch_size, raise gradient_accumulation_steps.

No GPU attached

Runtime > Change runtime type > T4 GPU, then re-run from the first cell.

ModuleNotFoundError

Re-run the pip install cell; on Colab, restart the session after installing unsloth.

© 2026 Tertiary Infotech Academy Pte Ltd

LU1 · HANDS-ON · SELF-PACED

Generate Synthetic Training Data

LAB 3

Generate synthetic fine-tuning examples that preserve the statistical properties of real data

Real data is scarce and privacy-restricted, so you generate synthetic examples using three different methods — template expansion, self-instruct bootstrapping and programmatic augmentation — then prove statistically that the synthetic set matches the real distribution.

YOU'LL BUILD

A synthetic dataset of several hundred examples produced by three methods, plus a distribution comparison report showing length, label-balance and vocabulary overlap against the real seed set.

TOOLCHAIN

Hugging Face datasets, pandas, numpy, scipy, matplotlib

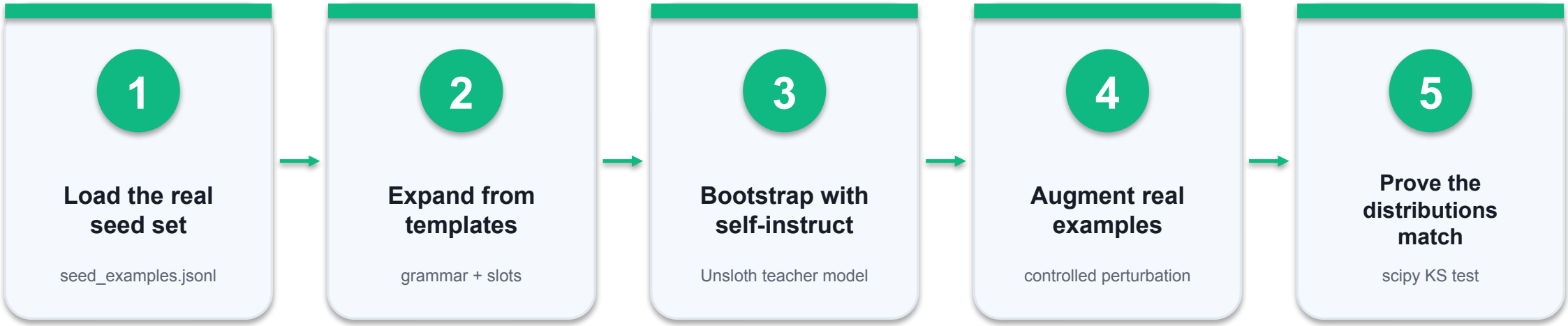
DONE WHEN

synthetic_examples.jsonl contains several hundred de-duplicated examples produced by all three methods, the KS test p-value is reported, vocabulary overlap is between roughly 40% and 95%, and synthetic_distribution.png shows the two length histograms overlapping.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 3 · THE SHAPE OF IT

How Lab 3 Runs



THE POINT

Generate synthetic fine-tuning examples that preserve the statistical properties of real data

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 3 · VERIFY

Generate Synthetic Training Data

✅ Expected result

synthetic_examples.jsonl contains several hundred de-duplicated examples produced by all three methods, the KS test p-value is reported, vocabulary overlap is between roughly 40% and 95%, and synthetic_distribution.png shows the two length histograms overlapping.

IF IT DOESN'T WORK

CUDA out of memory

Restart the runtime, lower per_device_train_batch_size, raise gradient_accumulation_steps.

No GPU attached

Runtime > Change runtime type > T4 GPU, then re-run from the first cell.

ModuleNotFoundError

Re-run the pip install cell; on Colab, restart the session after installing unsloth.

© 2026 Tertiary Infotech Academy Pte Ltd

LU1 · HANDS-ON · SELF-PACED

Ingest Datasets from Multiple Sources

LAB 4

Implement a dataset ingestion solution that loads, streams and merges heterogeneous sources

Build the ingestion layer of a fine-tuning pipeline: load from CSV, JSONL and the Hugging Face Hub, stream a dataset too large for RAM, normalise every source to one schema, and concatenate them into a single training corpus with provenance retained.

YOU'LL BUILD

A single unified DatasetDict assembled from four differently-shaped sources, each row tagged with its provenance, plus a streaming ingestion demonstration that never loads the full dataset.

TOOLCHAIN

Hugging Face datasets, pandas, JSON Lines, streaming IterableDataset

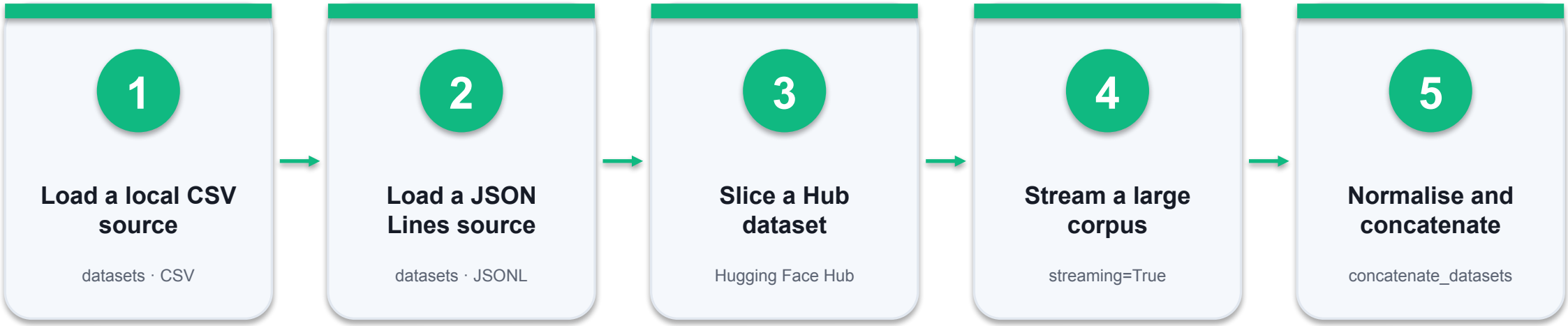
DONE WHEN

The merged corpus has exactly the four canonical columns, Counter(corpus['source']) shows rows from all three batch sources, the streaming demo prints three rows without a full download, and ingested_train.jsonl / ingested_test.jsonl are written.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 4 · THE SHAPE OF IT

How Lab 4 Runs



THE POINT

Implement a dataset ingestion solution that loads, streams and merges heterogeneous sources

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 4 · VERIFY

Ingest Datasets from Multiple Sources

✅ Expected result

The merged corpus has exactly the four canonical columns, Counter(corpus['source']) shows rows from all three batch sources, the streaming demo prints three rows without a full download, and ingested_train.jsonl / ingested_test.jsonl are written.

IF IT DOESN'T WORK

CUDA out of memory

Restart the runtime, lower per_device_train_batch_size, raise gradient_accumulation_steps.

No GPU attached

Runtime > Change runtime type > T4 GPU, then re-run from the first cell.

ModuleNotFoundError

Re-run the pip install cell; on Colab, restart the session after installing unsloth.

© 2026 Tertiary Infotech Academy Pte Ltd

LU1 · HANDS-ON · IN CLASS

Transform Data into Chat and Instruction Formats

LAB 5

Apply data transformation techniques that convert raw records into the model's training format

Convert the ingested corpus into the exact text the model must see during training. You render the Alpaca template by hand, render the model's native chat template with apply_chat_template(), append the EOS token, and prove why a template mismatch silently ruins a fine-tune.

YOU'LL BUILD

A formatted training dataset whose 'text' column is exactly what the trainer will tokenise, verified against the model's own chat template and confirmed to terminate with the EOS token.

TOOLCHAIN

Hugging Face datasets, transformers tokenizer, Unsloth chat templates

DONE WHEN

formatted_train.jsonl exists with a 'text' column; printing one row shows the Llama 3 header tokens; the token-length percentile table is produced; and no remaining example exceeds MAX_LEN tokens.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 5 · THE SHAPE OF IT

How Lab 5 Runs

1. **Load the ingested corpus**
   ingested_train.jsonl
2. **Render the Alpaca template**
   instruction format
3. **Render the native chat template**
   apply_chat_template
4. **Append the EOS token**
   tokenizer.eos_token
5. **Compare the two renderings**
   token ID inspection

THE POINT

Apply data transformation techniques that convert raw records into the model's training format

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 5 · VERIFY

Transform Data into Chat and Instruction Formats

✅ Expected result

formatted_train.jsonl exists with a 'text' column; printing one row shows the Llama 3 header tokens; the token-length percentile table is produced; and no remaining example exceeds MAX_LEN tokens.

IF IT DOESN'T WORK

CUDA out of memory

Restart the runtime, lower per_device_train_batch_size, raise gradient_accumulation_steps.

No GPU attached

Runtime > Change runtime type > T4 GPU, then re-run from the first cell.

ModuleNotFoundError

Re-run the pip install cell; on Colab, restart the session after installing unsloth.

© 2026 Tertiary Infotech Academy Pte Ltd

LU1 · HANDS-ON · SELF-PACED

Compare Generative Architectures and Accelerator Options

LAB 6

Compare GAN, diffusion and autoregressive architectures and select accelerator and cloud platform

Run a hands-on comparison of the three generative architecture families on the same small task, then measure the real memory and throughput cost of a fine-tune on the available accelerator so you can justify a hardware and cloud-platform choice with numbers rather than opinion.

YOU'LL BUILD

A comparison table of the three architecture families on generation quality, training stability and compute cost, plus measured throughput and VRAM figures for the lab GPU.

TOOLCHAIN

PyTorch, Unsloth, diffusers, matplotlib, nvidia-smi

DONE WHEN

architecture_comparison.csv and platform_comparison.csv are written, the autoregressive throughput is reported in tokens/second, the diffusion step-count timings show latency rising with steps, and peak VRAM is reported for both model types.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 6 · THE SHAPE OF IT

How Lab 6 Runs

1. **Time an autoregressive model**
   Unsloth · Llama 3.2
2. **Time a diffusion model**
   diffusers · tiny-sd
3. **Time a conditional GAN**
   PyTorch generator
4. **Measure peak VRAM each way**
   torch.cuda memory
5. **Build the comparison table**
   architecture table CSV

THE POINT

Compare GAN, diffusion and autoregressive architectures and select accelerator and cloud platform

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 6 · VERIFY

Compare Generative Architectures and Accelerator Options

✅ Expected result

architecture_comparison.csv and platform_comparison.csv are written, the autoregressive throughput is reported in tokens/second, the diffusion step-count timings show latency rising with steps, and peak VRAM is reported for both model types.

IF IT DOESN'T WORK

CUDA out of memory

Restart the runtime, lower per_device_train_batch_size, raise gradient_accumulation_steps.

No GPU attached

Runtime > Change runtime type > T4 GPU, then re-run from the first cell.

ModuleNotFoundError

Re-run the pip install cell; on Colab, restart the session after installing unsloth.

© 2026 Tertiary Infotech Academy Pte Ltd

Recap — Data Engineering

1

Lab 1

Prepare a reproducible GPU environment and verify the accelerator, VRAM and library stack

2

Lab 2

Build an immutable, versioned, reproducible data repository for a fine-tuning corpus

3

Lab 3

Generate synthetic fine-tuning examples that preserve the statistical properties of real data

4

Lab 4

Implement a dataset ingestion solution that loads, streams and merges heterogeneous sources

5

Lab 5

Apply data transformation techniques that convert raw records into the model's training format

6

Lab 6

Compare GAN, diffusion and autoregressive architectures and select accelerator and cloud platform

© 2026 Tertiary Infotech Academy Pte Ltd

Common Mistakes — Data Engineering

1

Training on the wrong template

The single biggest cause of a fine-tune that trains cleanly and behaves badly. Always use tokenizer.apply_chat_template().

2

Forgetting the EOS token

The model never learns where an answer ends and generates until it hits the token limit.

3

An unseeded split

An unseeded train/test split cannot be reproduced, which silently invalidates every later comparison.

4

Synthetic data that is a paraphrase

If it merely restates the seed set it adds no coverage — de-duplicate against the real data.

5

No provenance on merged sources

When quality drops you cannot tell which source caused it.

6

Ignoring the token-length distribution

Truncated examples teach the model to produce unfinished answers.

© 2026 Tertiary Infotech Academy Pte Ltd

02

LEARNING UNIT 02

Exploratory Data Analysis

Cleaning and sanitisation · optimisation strategies for higher-quality outputs · feature engineering · ETL pipelines · analysis and visualisation

© 2026 Tertiary Infotech Academy Pte Ltd

LU2 at a Glance

1

Learning outcome

Develop optimized data pipelines for feature engineering using various optimization strategies.

2

Hands-on labs

Lab 7 in class · Labs 8–12 self-paced

3

Weighting

25% of course time · Late morning

4

Topics

Sanitize and prepare data for modelling · Perform feature engineering for fine tuning · Analyze and visualize data for fine tuning insights

© 2026 Tertiary Infotech Academy Pte Ltd

Key Concepts — Exploratory Data Analysis

1. Garbage in, garbage out is literal for fine tuning: a 1,000-example clean dataset routinely beats a 100,000-example dirty one, because LoRA has few parameters and will faithfully learn whatever noise you show it.

2. Sanitisation removes PII, exact and near duplicates, truncated records, prompt-injection payloads and toxic content — each of which becomes a learned behaviour if left in.

3. MinHash/LSH near-duplicate detection catches the paraphrased copies that exact hashing misses, and must also be run across the train/test boundary to prevent evaluation leakage.

4. Optimisation strategies for higher-quality outputs: curate for quality over quantity, balance the task mix, tune decoding parameters, apply preference optimisation (DPO), and iterate with error analysis rather than blindly adding data.

© 2026 Tertiary Infotech Academy Pte Ltd

LU2 · WEIGHTING 25%

Key Concepts — Exploratory Data Analysis (2)

1. Decoding parameters — temperature, top-p, top-k, repetition penalty and max_new_tokens — change output quality at inference with zero training cost, and must be held fixed when comparing models.

2. Feature engineering for LLM fine tuning is prompt-side engineering: system-prompt design, few-shot exemplar selection, field concatenation order, length bucketing and difficulty tagging.

3. Feature selection here means example selection — perplexity filtering, embedding-diversity sampling and quality scoring keep the highest-signal subset of a large candidate pool.

4. Data augmentation for generative models: paraphrase generation, back-translation, instruction rewriting, and response re-ranking expand a small seed set without new human labelling.

© 2026 Tertiary Infotech Academy Pte Ltd

Key Concepts — Exploratory Data Analysis (3)

1. An ETL pipeline for a fine-tuning dataset is Extract (load raw sources) → Transform (clean, dedupe, template, tokenise) → Load (push a versioned, split dataset), with a validation gate between each stage.

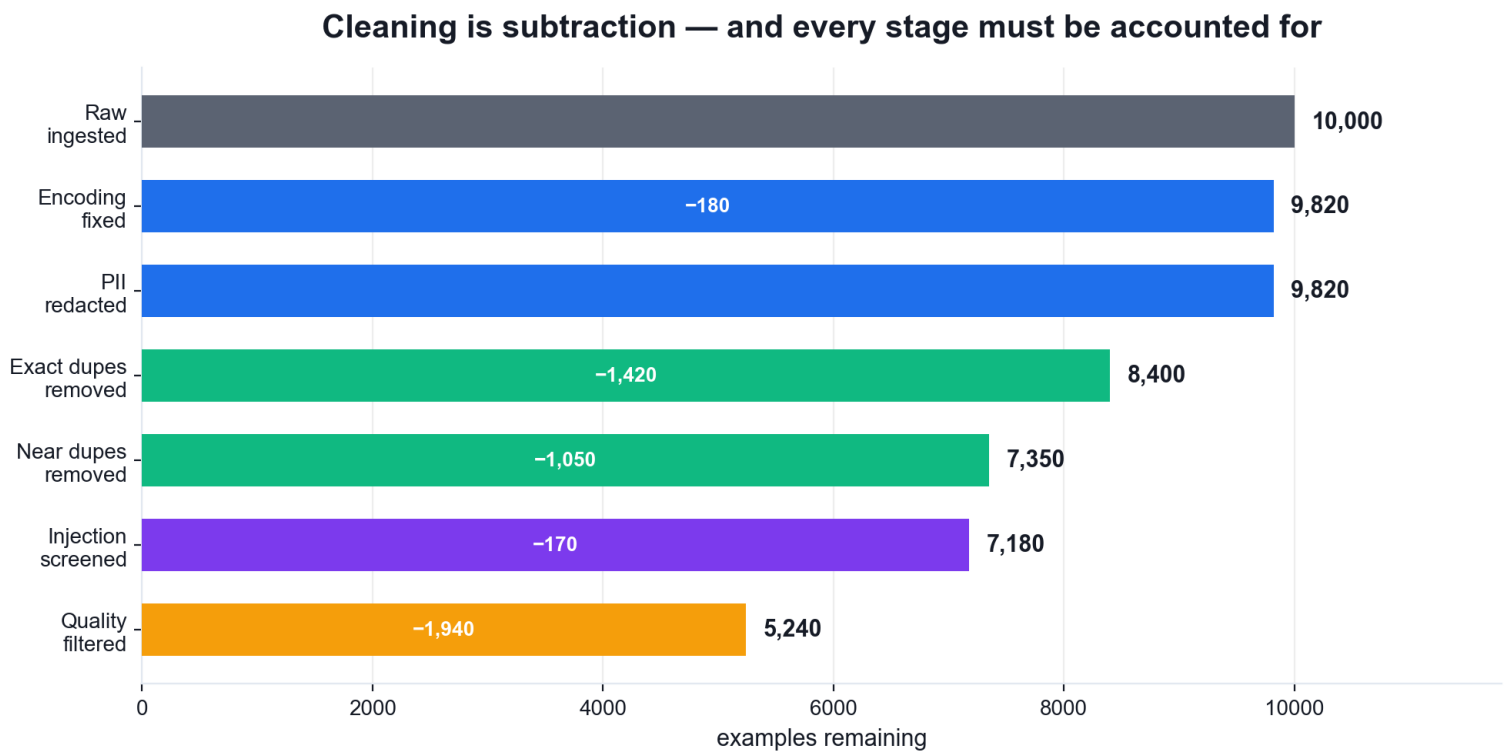
2. Generative concepts: transfer learning reuses pre-trained representations, knowledge distillation trains a small student from a large teacher, self-supervised learning creates labels from the data itself, and explainability makes the resulting behaviour inspectable.

3. Token-length analysis drives the `max_seq_length` choice: set it at roughly the 95th–99th percentile so you neither truncate real content nor pay quadratic attention cost for padding.

4. Visualising label balance, length distribution and embedding clusters (UMAP/t-SNE) exposes the gaps and redundancies in a dataset before you spend GPU hours on it.

© 2026 Tertiary Infotech Academy Pte Ltd

Cleaning Is Subtraction



A 52% retention rate is normal. What matters is that cleaning_report.json accounts for every removed row — an unexplained drop is a bug.

A 50% retention rate is normal. What matters is that every removed row is accounted for — an unexplained drop is a bug, not a cleaning success.

© 2026 Tertiary Infotech Academy Pte Ltd

What Must Come Out of a Training Set

1. **PII**
   Emails, phones, NRIC/FIN, card numbers — a fine-tuned model memorises and regurgitates them.

2. **Exact duplicates**
   Implicit over-weighting: the model sees them more often and over-fits to them.

3. **Near duplicates**
   Paraphrased copies survive exact hashing and leak evaluation data into training.

4. **Truncated records**
   Teach the model to produce unfinished answers.

5. **Prompt injection**
   A poisoned example teaches the model to obey the injection — a supply-chain attack.

6. **Encoding damage**
   Mojibake becomes learned garbage tokens.

© 2026 Tertiary Infotech Academy Pte Ltd

LU2 · QUALITY OVER QUANTITY

A thousand clean examples beat a hundred thousand dirty ones.

LoRA has very few parameters, so it learns whatever you show it with high fidelity — including the noise. Curation is not preparation for the real work; it IS the work.

© 2026 Tertiary Infotech Academy Pte Ltd

Feature Engineering for Fine Tuning

1

System prompt design

The highest-leverage single feature: fixes role, tone and refusal behaviour across every example.

2

Few-shot exemplar selection

Attach the nearest neighbours as in-context examples to teach the format faster.

3

Field ordering

Instruction, then context, then input — a consistent order the model can rely on.

4

Length bucketing

Group similar lengths into a batch to cut padding waste — free throughput.

5

Difficulty tagging

Control the curriculum mix, and diagnose which band the model fails on.

6

Example selection

Perplexity filtering and embedding diversity keep the highest-signal subset.

© 2026 Tertiary Infotech Academy Pte Ltd

The ETL Pipeline for a Fine-Tuning Dataset



WHAT MAKES IT A PIPELINE

The quality gate. A script transforms data; a pipeline refuses to ship data that violates its declared contract.

© 2026 Tertiary Infotech Academy Pte Ltd

Generative Concepts You Must Explain

1

Transfer learning

Pre-trained representations are reused; fine tuning adapts rather than teaching from scratch.

2

Knowledge distillation

A large teacher supplies soft targets that transfer capability to a smaller student.

3

Self-supervised learning

Labels come from the data itself — next-token prediction created the base model.

4

Explainability

Making the resulting behaviour inspectable: attribution, probing, ablation.

© 2026 Tertiary Infotech Academy Pte Ltd

Decoding Parameters — Free Quality

Parameter	What it does	Low value	High value	Use it for
temperature	Rescales logits before sampling	Focused, repeatable	Creative, incoherent	Tone of output
top_p	Samples from the top probability mass	Conservative	Full distribution	Adaptive diversity
top_k	Restricts to the k likeliest tokens	Very narrow	Little effect	Hard cap on candidates
repetition_penalty	Penalises already-used tokens	Loops appear	Unnatural phrasing	Killing 4-gram loops

WHEN IT MATTERS Use GREEDY decoding for every evaluation so results are reproducible, and sampling only for user-facing text. Never compare two models under different decoders.

© 2026 Tertiary Infotech Academy Pte Ltd

WHAT YOU'LL DO

Hands-On Labs — Exploratory Data Analysis

Labs 7–8

- Clean and Sanitize a Fine-Tuning Dataset
- Feature Engineering for Fine Tuning

Labs 9–10

- Build an ETL Pipeline for Fine-Tuning Datasets
- Analyse and Visualise a Fine-Tuning Dataset

Labs 11–12

- Tune Decoding Parameters for Higher-Quality Output
- Transfer Learning, Distillation and Self-Supervised Learning

© 2026 Tertiary Infotech Academy Pte Ltd

LU2 · HANDS-ON · IN CLASS

Clean and Sanitize a Fine-Tuning Dataset

LAB 7

Clean, sanitize and prepare a raw corpus so the fine-tune learns signal rather than noise

Take a deliberately dirty corpus and run the full sanitisation gauntlet — PII redaction, exact and near-duplicate removal, truncation and encoding repair, prompt-injection screening and quality scoring — measuring how many rows each stage removes and why.

YOU'LL BUILD

A cleaned dataset plus a cleaning report that records, stage by stage, how many rows were removed and for what reason — the audit trail a data pipeline must produce.

TOOLCHAIN

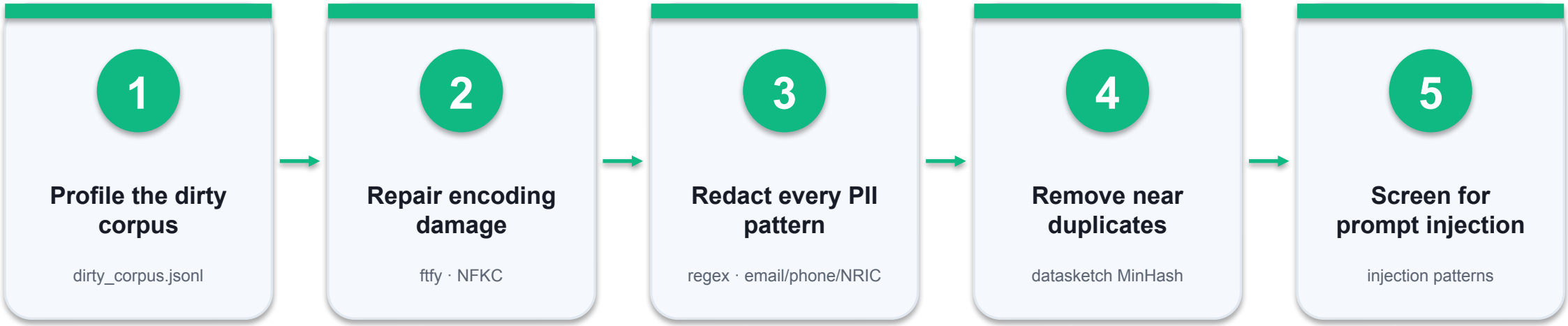
pandas, regex, datasketch MinHash/LSH, ftfy, Hugging Face datasets

DONE WHEN

cleaned_corpus.jsonl exists, cleaning_report.json accounts for every removed row by stage, no email/phone/NRIC pattern survives a re-scan of the cleaned file, and retention_pct is reported.

© 2026 Tertiary Infotech Academy Pte Ltd

How Lab 7 Runs



THE POINT

Clean, sanitize and prepare a raw corpus so the fine-tune learns signal rather than noise

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 7 · VERIFY

Clean and Sanitize a Fine-Tuning Dataset

✅ Expected result

cleaned_corpus.jsonl exists, cleaning_report.json accounts for every removed row by stage, no email/phone/NRIC pattern survives a re-scan of the cleaned file, and retention_pct is reported.

IF IT DOESN'T WORK

CUDA out of memory

Restart the runtime, lower per_device_train_batch_size, raise gradient_accumulation_steps.

No GPU attached

Runtime > Change runtime type > T4 GPU, then re-run from the first cell.

ModuleNotFoundError

Re-run the pip install cell; on Colab, restart the session after installing unsloth.

© 2026 Tertiary Infotech Academy Pte Ltd

LU2 · HANDS-ON · SELF-PACED

Feature Engineering for Fine Tuning

LAB 8

Apply feature engineering, feature selection and data augmentation to a fine-tuning set

Engineer the prompt-side features that decide fine-tuning quality: system-prompt design, few-shot exemplar selection, field ordering, length bucketing and difficulty tagging — then apply embedding-diversity feature selection and three augmentation strategies.

YOU'LL BUILD

An engineered dataset carrying system prompts, difficulty tags and length buckets, reduced by diversity-based selection and expanded by augmentation, with before/after coverage metrics.

TOOLCHAIN

sentence-transformers, scikit-learn, pandas, numpy, Hugging Face datasets

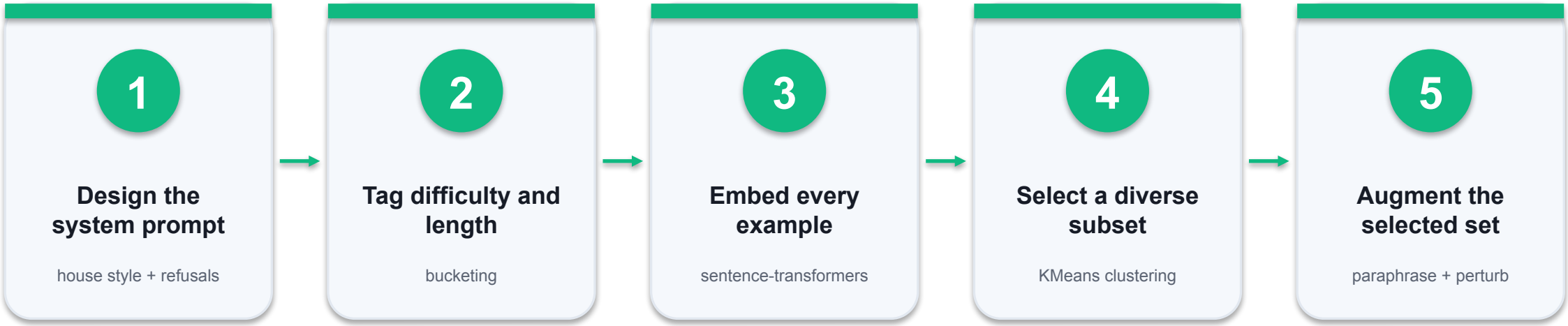
DONE WHEN

engineered_corpus.jsonl contains the system, difficulty and bucket feature columns; the diversity score of the selected subset is higher than the random subset; and the padding-waste saving from bucketing is reported as a percentage.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 8 · THE SHAPE OF IT

How Lab 8 Runs



THE POINT

Apply feature engineering, feature selection and data augmentation to a fine-tuning set

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 8 · VERIFY

Feature Engineering for Fine Tuning

✅ Expected result

engineered_corpus.jsonl contains the system, difficulty and bucket feature columns; the diversity score of the selected subset is higher than the random subset; and the padding-waste saving from bucketing is reported as a percentage.

IF IT DOESN'T WORK

CUDA out of memory

Restart the runtime, lower per_device_train_batch_size, raise gradient_accumulation_steps.

No GPU attached

Runtime > Change runtime type > T4 GPU, then re-run from the first cell.

ModuleNotFoundError

Re-run the pip install cell; on Colab, restart the session after installing unsloth.

© 2026 Tertiary Infotech Academy Pte Ltd

LU2 · HANDS-ON · SELF-PACED

Build an ETL Pipeline for Fine-Tuning Datasets

LAB 9

Develop an extract-transform-load pipeline that produces a validated fine-tuning dataset

Assemble the previous labs' work into one orchestrated, re-runnable ETL pipeline with validation gates between stages, structured logging, failure handling and an output contract — the form a data pipeline must take before it can run in production.

YOU'LL BUILD

A single run_pipeline.py that goes from raw sources to a versioned, validated, split dataset, with a quality gate that fails the run rather than shipping a bad dataset.

TOOLCHAIN

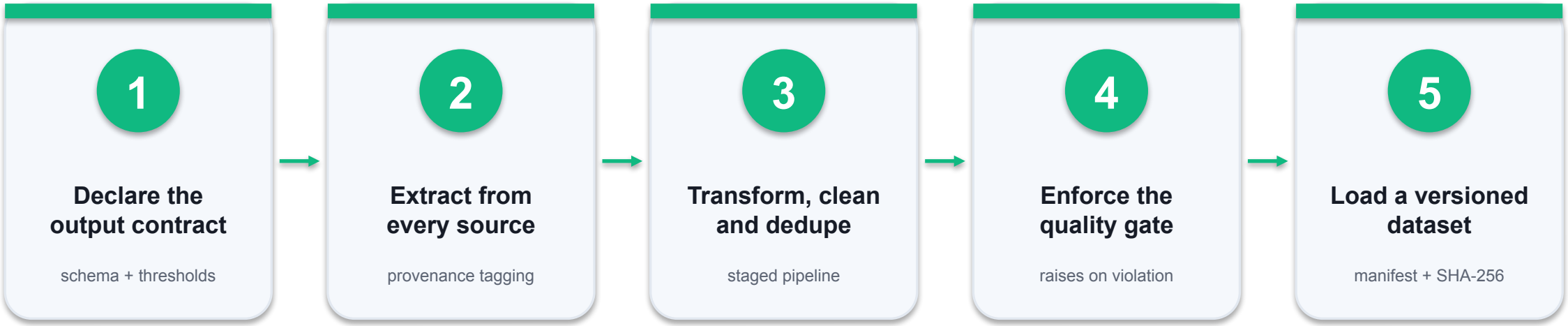
Hugging Face datasets, pandas, Python logging, JSON Schema

DONE WHEN

The pipeline runs end to end, the quality gate passes on the clean data and raises ValueError on the deliberately poisoned copy, data_repo/v2/manifest.json records per-stage statistics and a hash per split, and the stage report shows retention at every stage.

© 2026 Tertiary Infotech Academy Pte Ltd

How Lab 9 Runs



THE POINT

Develop an extract-transform-load pipeline that produces a validated fine-tuning dataset

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 9 · VERIFY

Build an ETL Pipeline for Fine-Tuning Datasets

✅ Expected result

The pipeline runs end to end, the quality gate passes on the clean data and raises ValueError on the deliberately poisoned copy, data_repo/v2/manifest.json records per-stage statistics and a hash per split, and the stage report shows retention at every stage.

IF IT DOESN'T WORK

CUDA out of memory

Restart the runtime, lower per_device_train_batch_size, raise gradient_accumulation_steps.

No GPU attached

Runtime > Change runtime type > T4 GPU, then re-run from the first cell.

ModuleNotFoundError

Re-run the pip install cell; on Colab, restart the session after installing unsloth.

© 2026 Tertiary Infotech Academy Pte Ltd

LU2 · HANDS-ON · SELF-PACED

Analyse and Visualise a Fine-Tuning Dataset

LAB 10

Analyse and visualise dataset properties to drive fine-tuning decisions

Profile the prepared dataset before spending GPU hours on it: token-length distributions that set max_seq_length, task balance, vocabulary coverage, and a UMAP embedding map that exposes topic clusters and coverage gaps.

YOU'LL BUILD

A dataset profile report with five diagnostic plots and a written set of concrete recommendations (max_seq_length, rebalancing, gaps to fill) grounded in the measurements.

TOOLCHAIN

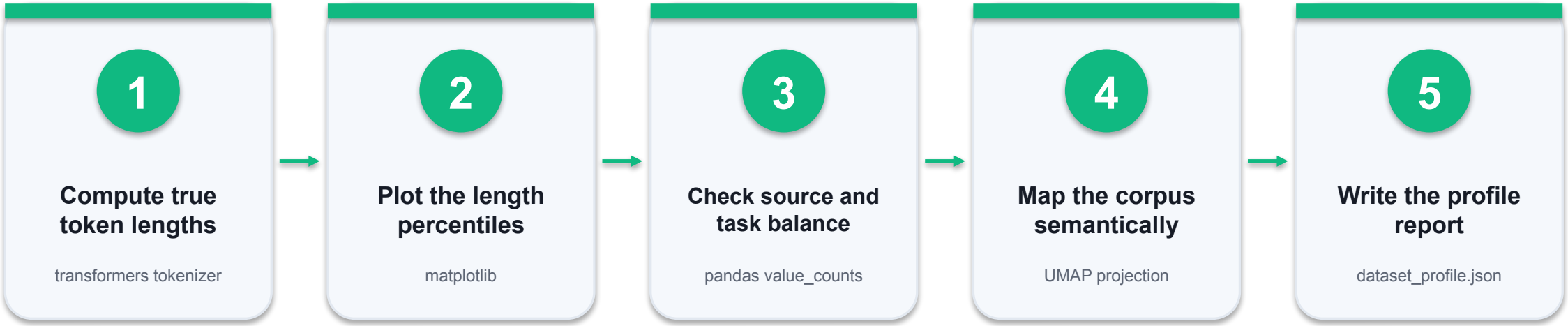
matplotlib, seaborn, UMAP, sentence-transformers, pandas, transformers tokenizer

DONE WHEN

profile_lengths.png, profile_balance.png and profile_umap.png are produced; the token-length percentile table yields a recommended max_seq_length; the eight clusters are each labelled with their top terms; and dataset_profile.json carries the numeric recommendations.

© 2026 Tertiary Infotech Academy Pte Ltd

How Lab 10 Runs



THE POINT

Analyse and visualise dataset properties to drive fine-tuning decisions

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 10 · VERIFY

Analyse and Visualise a Fine-Tuning Dataset

✅ Expected result

profile_lengths.png, profile_balance.png and profile_umap.png are produced; the token-length percentile table yields a recommended max_seq_length; the eight clusters are each labelled with their top terms; and dataset_profile.json carries the numeric recommendations.

IF IT DOESN'T WORK

CUDA out of memory

Restart the runtime, lower per_device_train_batch_size, raise gradient_accumulation_steps.

No GPU attached

Runtime > Change runtime type > T4 GPU, then re-run from the first cell.

ModuleNotFoundError

Re-run the pip install cell; on Colab, restart the session after installing unsloth.

© 2026 Tertiary Infotech Academy Pte Ltd

LU2 · HANDS-ON · SELF-PACED

Tune Decoding Parameters for Higher-Quality Output

LAB 11

Apply optimisation strategies that raise output quality without retraining

Before spending money on training, extract the quality that is already available for free: sweep temperature, top-p, top-k and repetition penalty on a fixed prompt set, measure the effect on diversity and repetition, and lock in an evaluation-time decoding configuration.

YOU'LL BUILD

A decoding parameter sweep with measured diversity, repetition and length metrics, and a recommended generation_config.json used unchanged by every later evaluation lab.

TOOLCHAIN

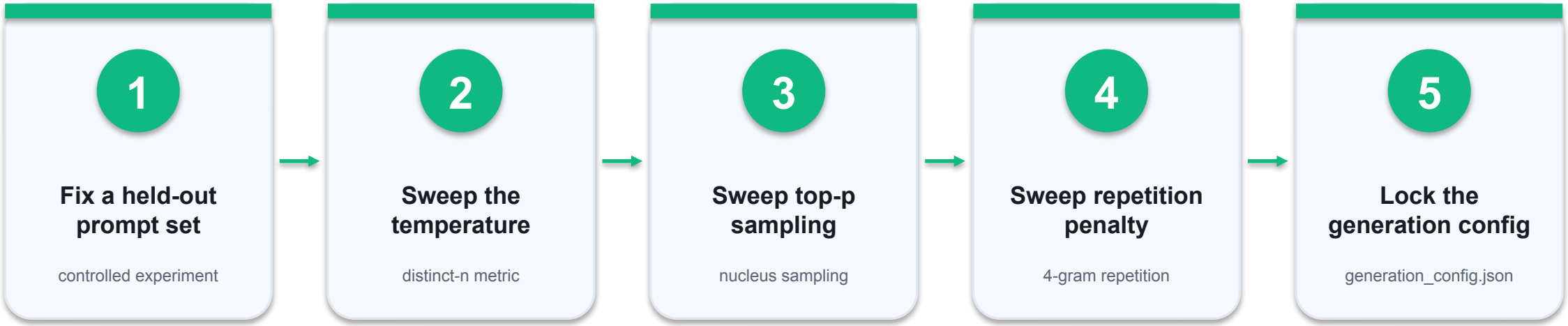
Unsloth, transformers GenerationConfig, pandas, matplotlib

DONE WHEN

decoding_sweep.png shows all three parameter sweeps; the temperature table shows distinct-2 rising with temperature; the repetition-penalty table shows the repetition rate falling; and generation_config.json holds both the evaluation and production configurations.

© 2026 Tertiary Infotech Academy Pte Ltd

How Lab 11 Runs



THE POINT

Apply optimisation strategies that raise output quality without retraining

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 11 · VERIFY

Tune Decoding Parameters for Higher-Quality Output

✅ Expected result

decoding_sweep.png shows all three parameter sweeps; the temperature table shows distinct-2 rising with temperature; the repetition-penalty table shows the repetition rate falling; and generation_config.json holds both the evaluation and production configurations.

IF IT DOESN'T WORK

CUDA out of memory

Restart the runtime, lower per_device_train_batch_size, raise gradient_accumulation_steps.

No GPU attached

Runtime > Change runtime type > T4 GPU, then re-run from the first cell.

ModuleNotFoundError

Re-run the pip install cell; on Colab, restart the session after installing unsloth.

© 2026 Tertiary Infotech Academy Pte Ltd

LU2 · HANDS-ON · SELF-PACED

Transfer Learning, Distillation and Self-Supervised Learning

LAB 12

Explain and demonstrate the generative concepts that underpin fine tuning

Make the four foundational generative concepts concrete: measure what transfer learning already gives you, distil a large teacher into a small student, run a self-supervised masked objective, and inspect the model to show what 'explainability' means in practice.

YOU'LL BUILD

Four short empirical demonstrations — a transfer-learning comparison, a distillation dataset with measured agreement, a self-supervised training loop, and a token-attribution plot.

TOOLCHAIN

PyTorch, Unsloth, transformers, matplotlib, numpy

DONE WHEN

The domain text scores a clearly lower perplexity than the shuffled text; distilled_dataset.jsonl holds the teacher's generations; the softmax temperature sweep shows the distribution flattening; explainability_attribution.png is produced; and generative_concepts.json records all four.

© 2026 Tertiary Infotech Academy Pte Ltd

How Lab 12 Runs

1. **Measure transfer learning**
   perplexity comparison
2. **Distil a teacher model**
   soft targets
3. **Inspect the soft distribution**
   softmax temperature
4. **Run the self-supervised loss**
   next-token shift
5. **Attribute the prediction**
   gradient x input

THE POINT

Explain and demonstrate the generative concepts that underpin fine tuning

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 12 · VERIFY

Transfer Learning, Distillation and Self-Supervised Learning

✅ Expected result

The domain text scores a clearly lower perplexity than the shuffled text; distilled_dataset.jsonl holds the teacher's generations; the softmax temperature sweep shows the distribution flattening; explainability_attribution.png is produced; and generative_concepts.json records all four.

IF IT DOESN'T WORK

CUDA out of memory

Restart the runtime, lower per_device_train_batch_size, raise gradient_accumulation_steps.

No GPU attached

Runtime > Change runtime type > T4 GPU, then re-run from the first cell.

ModuleNotFoundError

Re-run the pip install cell; on Colab, restart the session after installing unsloth.

© 2026 Tertiary Infotech Academy Pte Ltd

Recap — Exploratory Data Analysis

1

Lab 7

Clean, sanitize and prepare a raw corpus so the fine-tune learns signal rather than noise

2

Lab 8

Apply feature engineering, feature selection and data augmentation to a fine-tuning set

3

Lab 9

Develop an extract-transform-load pipeline that produces a validated fine-tuning dataset

4

Lab 10

Analyse and visualise dataset properties to drive fine-tuning decisions

5

Lab 11

Apply optimisation strategies that raise output quality without retraining

6

Lab 12

Explain and demonstrate the generative concepts that underpin fine tuning

© 2026 Tertiary Infotech Academy Pte Ltd

Common Mistakes — Exploratory Data Analysis

1

Cleaning without a report

If you cannot account for every removed row, an unexplained drop is a bug you will never find.

2

Only removing exact duplicates

Near-duplicates survive exact hashing and leak evaluation examples into training.

3

Redacting PII after splitting

Redact BEFORE the split, or leaked values persist in one split only.

4

Adding data instead of fixing data

More dirty examples make the problem worse, not better.

5

Comparing models under different decoders

A temperature difference alone can flip which model looks better.

6

Setting max_seq_length by guesswork

Measure the 99th percentile — too short truncates, too long wastes quadratic attention cost.

© 2026 Tertiary Infotech Academy Pte Ltd

03

LEARNING UNIT 03

Modelling

Framing the fine-tuning problem · multi-modal embeddings · LoRA and QLoRA · Unsloth · training and monitoring · hyperparameter optimisation · advanced loss metrics

© 2026 Tertiary Infotech Academy Pte Ltd

LU3 at a Glance

1

Learning outcome

Fine-tune pre-trained multi-modal models using advanced loss metrics and training strategies.

2

Hands-on labs

Labs 15 and 17 in class · Labs 13, 14, 16 and 18–20 self-paced

3

Weighting

30% of course time · Afternoon

4

Topics

Frame business problems as fine-tuning problems · Select the appropriate model and adaptation method · Train and fine-tune models with LoRA, QLoRA and Unsloth · Perform hyperparameter optimization · Evaluate fine-tuned models

© 2026 Tertiary Infotech Academy Pte Ltd

Key Concepts — Modelling

1. Not every problem is a fine-tuning problem: prompt engineering fixes instruction-following, RAG fixes missing knowledge, and fine tuning fixes behaviour, format, tone and domain style. Choosing wrongly wastes the budget.

2. LoRA freezes the pre-trained weight matrix W and learns a low-rank update: h = Wx + (alpha/r)·BAx, where A is r x k, B is d x r, and only A and B are trained — typically under 1% of parameters.

3. B is initialised to zeros and A to a Kaiming/Gaussian draw, so BA = 0 at step 0 and the adapted model starts exactly equal to the base model — training is a strictly additive perturbation.

4. The rank r sets adapter capacity (8-16 for style and format, 32-64 for genuinely new domain skills); lora_alpha sets the update strength through the alpha/r scaling factor.

© 2026 Tertiary Infotech Academy Pte Ltd

Key Concepts — Modelling (2)

1. Rank-stabilised LoRA (use_rslora) scales by alpha/sqrt(r) instead of alpha/r, which keeps the update magnitude stable as the rank is raised.

2. target_modules decides where adaptation happens. Attention-only (q_proj, v_proj) is cheapest but underfits; the Databricks study showed that targeting ALL linear layers — attention plus the MLP gate_proj, up_proj and down_proj — is what produces high-quality output.

3. QLoRA adds 4-bit NF4 quantisation of the frozen base weights, double quantisation and paged optimisers, cutting VRAM roughly 3-4x again so a 7B-8B model fine-tunes on a single 16GB GPU.

4. QLoRA computes in bf16/fp16 while storing in 4-bit: the frozen weights are dequantised on the fly for each matmul, so quality stays close to 16-bit LoRA while memory falls dramatically.

© 2026 Tertiary Infotech Academy Pte Ltd

LU3 · WEIGHTING 30%

Key Concepts — Modelling (3)

1. Unsloth rewrites the attention, MLP and cross-entropy kernels in Triton and hand-derives the backward pass, delivering roughly 2x faster training with about 70-80% less VRAM and no loss of accuracy versus a standard PEFT + Transformers loop.

2. Multi-modal embeddings place text, image and audio into one shared vector space through a contrastive objective (CLIP-style), so a text query can retrieve or condition on an image.

3. A multi-modal model joins a vision encoder to a language model through a projector; fine tuning usually freezes the vision tower and adapts the projector and the LLM with LoRA.

4. Advanced loss functions and metrics: cross-entropy and perplexity for language modelling, FID and Inception Score for images, CLIPScore for text-image alignment, BLEU/ROUGE/BERTScore for text similarity, and adversarial/contrastive/diffusion losses for their respective architectures.

© 2026 Tertiary Infotech Academy Pte Ltd

Key Concepts — Modelling (4)

1. Monitoring stability and convergence: a healthy run shows training loss falling smoothly, validation loss tracking it, gradient norm bounded and learning rate following its schedule.

2. Divergence signatures: a loss spike to NaN means the learning rate is too high or fp16 overflowed; validation loss rising while training loss falls is overfitting; a flat loss means the learning rate is too low, the adapter is not attached, or labels are masked out.

3. Hyperparameter optimisation has a clear priority order: learning rate first (1e-4 to 2e-4 for LoRA — roughly 10x a full fine-tune), then epochs (1-3), then r and alpha, then batch size via gradient accumulation.

4. Effective batch size = per_device_batch_size x gradient_accumulation_steps x num_devices; accumulation buys a large effective batch on small VRAM at the cost of wall-clock time.

© 2026 Tertiary Infotech Academy Pte Ltd

Key Concepts — Modelling (5)

1. Intermediate fine tuning adapts the base model on a large related corpus BEFORE the small task set; continuous pre-training keeps the self-supervised objective on new raw domain text to add knowledge rather than behaviour.

2. Merging (`merge_and_unload`) folds BA back into W, giving a standalone model with exactly zero added inference latency; keeping adapters separate instead lets one base model serve many tasks.

© 2026 Tertiary Infotech Academy Pte Ltd

How a LoRA Fine-Tune Actually Runs



WHY IT IS CHEAP

The optimiser state — usually the biggest memory consumer — exists only for the adapter, not for the billions of frozen base parameters.

© 2026 Tertiary Infotech Academy Pte Ltd

Reading a Training Run



Four signatures you must be able to name on sight. Diagnosis comes before any change to the configuration.

© 2026 Tertiary Infotech Academy Pte Ltd

Training Failure Modes and Their Fixes

Symptom	Likely cause	The fix
Loss → NaN or Inf	LR too high, or fp16 overflow	Lower LR 10×, use bf16, clip grad to 1.0
Loss spikes then recovers	Bad batch or LR slightly high	Clip gradients, add warmup steps
Loss completely flat	LR too low / adapter not attached	Raise LR; check print_trainable_parameters
Val loss rises, train falls	Overfitting	Fewer epochs, early stopping, more data
Gradient norm > 10	Unstable updates	max_grad_norm = 1.0, lower the LR
CUDA out of memory	Batch or sequence too large	Lower batch, raise grad accumulation

WHEN IT MATTERS Diagnose from the CURVE, not from the final number. A single loss value cannot distinguish overfitting from healthy convergence.

© 2026 Tertiary Infotech Academy Pte Ltd

LU3 · TUNE IN THIS ORDER

Hyperparameter Priority Order

1

1 · Learning rate

By far the most sensitive. 1e-4 to 2e-4 for LoRA — roughly 10× a full fine-tune.

2

2 · Epochs

1–3 for most datasets. More epochs on a small set buys overfitting, not quality.

3

3 · Rank r and alpha

r = 8–16 for style; 32–64 for new skills. alpha = 2r is a sound default.

4

4 · target_modules

All linear layers. This is a quality decision more than a cost decision.

5

5 · Effective batch size

batch × grad_accum × devices. Accumulation buys a large batch on small VRAM.

6

6 · Everything else

Warmup, weight decay, scheduler — these rarely decide the outcome.

© 2026 Tertiary Infotech Academy Pte Ltd

LU3 · WORKED EXAMPLE

The Training Configuration, Line by Line

These are the settings you will tune in the labs — know why each value is what it is.

```
args = SFTConfig(
    per_device_train_batch_size = 2,
    gradient_accumulation_steps = 4,   # eff. batch 8
    num_train_epochs            = 2,
    learning_rate               = 2e-4,
    warmup_steps                = 5,
    optim             = 'adamw_8bit',
    lr_scheduler_type = 'linear',
    weight_decay      = 0.01,
    max_grad_norm     = 1.0,
    bf16 = is_bfloat16_supported(),
    seed = 3407,
)
```

2e-4 learning rate

~10× a full fine-tune, because the adapter is small and starts from zero.

Effective batch 8

batch 2 × accumulation 4. Accumulation trades wall-clock time for VRAM.

adamw_8bit

Quantised optimiser state — a large memory saving for no measurable quality cost.

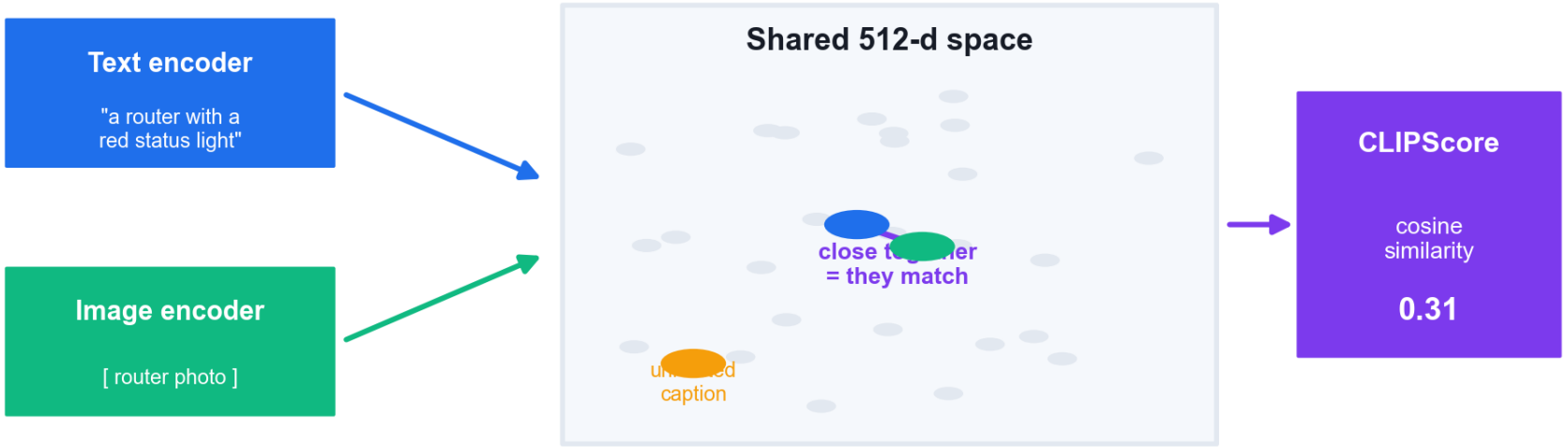
max_grad_norm 1.0

Gradient clipping: the single most effective guard against divergence.

© 2026 Tertiary Infotech Academy Pte Ltd

Multi-Modal Embeddings

Multi-modal embeddings — one shared vector space



WHY IT MATTERS
Because text and images land in the SAME space, a text query can retrieve an image, an image can condition a generator, and CLIPScore can grade text–image alignment as a training metric.

A contrastive objective places text and images in one vector space, so a text query can retrieve an image and CLIPScore can grade alignment.

© 2026 Tertiary Infotech Academy Pte Ltd

Which Metric for Which Generative Task

Task	Loss	Metric	Direction	The catch
Language modelling	Cross-entropy	Perplexity	lower	Blind to factual accuracy
Translation	Cross-entropy	BLEU / chrF	higher	Punishes valid paraphrase
Summarisation	Cross-entropy	ROUGE-1/2/L	higher	Recall-biased; rewards copying
Open-ended text	Cross-entropy	BERTScore + LLM judge	higher	Judge needs human validation
Image generation	Diffusion MSE	FID	lower	Needs thousands of samples
Text-to-image	Diffusion MSE	CLIPScore	higher	Alignment, not image quality

WHEN IT MATTERS Using an invalid metric for the task is the most common evaluation error. Perplexity in particular can rate a fluent-but-wrong answer above a correct one.

© 2026 Tertiary Infotech Academy Pte Ltd

Three Objectives Behind the Three Architectures

1

Cross-entropy

Autoregressive models. Maximise the likelihood of the next token. Stable, well understood.

2

Adversarial loss

GANs. Generator and discriminator play a minimax game — powerful but hard to balance.

3

Diffusion MSE

Diffusion models. Predict the noise that was added. One simple loss, hence very stable.

4

InfoNCE contrastive

CLIP-style multi-modal models. Pull matched pairs together, push others apart.

© 2026 Tertiary Infotech Academy Pte Ltd

Intermediate Fine Tuning vs Continuous Pre-Training

Strategy	Objective	Data	What it adds	Use when
Task fine tuning	Supervised	Small labelled set	Behaviour, format, style	You have target examples
Intermediate fine tuning	Supervised ×2	Large related, then task	Domain competence first	Task data is scarce
Continuous pre-training	Self-supervised	Large RAW domain text	Domain knowledge, vocabulary	The domain language is unfamiliar

WHEN IT MATTERS The recommended sequence when all three are available: continuous pre-training, then intermediate fine tuning, then task fine tuning on the small target set.

© 2026 Tertiary Infotech Academy Pte Ltd

WHAT YOU'LL DO

Hands-On Labs — Modelling

Labs 13–15

- Frame the Problem and Select the Adaptation Method
- Build LoRA from First Principles
- Fine-Tune with PEFT LoRA

Labs 16–18

- Fine-Tune with QLoRA in 4-Bit
- Accelerate Fine Tuning with Unsloth
- Monitor Training Stability and Convergence

Labs 19–20

- Hyperparameter Optimization for LoRA
- Advanced Loss Functions and Generative Metrics

© 2026 Tertiary Infotech Academy Pte Ltd

LU3 · HANDS-ON · SELF-PACED

Frame the Problem and Select the Adaptation Method

LAB 13

Frame a business problem as a fine-tuning problem and justify the adaptation method

Before touching a GPU, decide whether the problem needs prompt engineering, RAG or fine tuning. You test all three on the same business case, measure the result, and produce a written justification — including the multi-modal embedding case where the input is not text alone.

YOU'LL BUILD

A decision record that tests prompting, RAG and fine tuning on one problem with measured results, plus a multi-modal embedding demonstration showing when a shared vector space is needed.

TOOLCHAIN

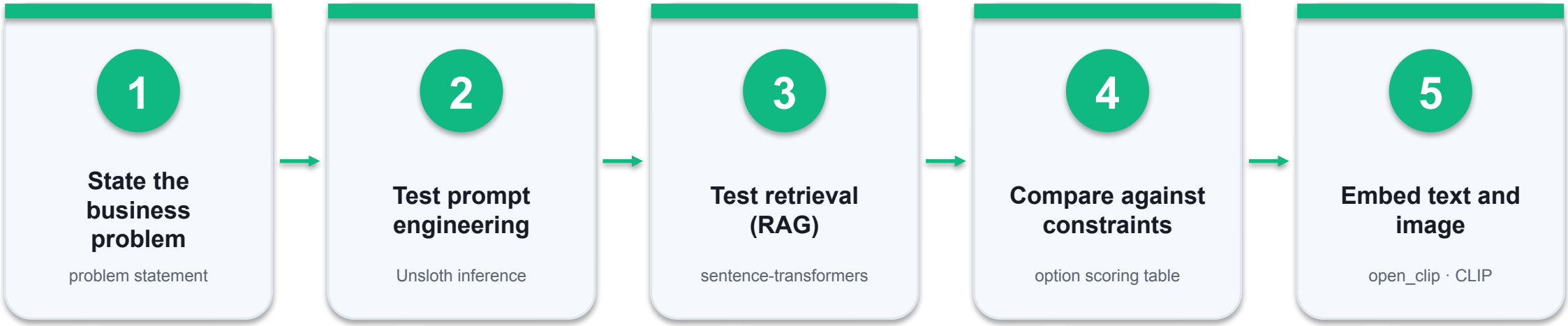
Unsloth, sentence-transformers, CLIP, scikit-learn, pandas

DONE WHEN

All three options produce output on the same ticket, the option comparison table is printed, CLIP scores the correct caption highest, and adaptation_decision.json records the chosen method with an explicit justification against the stated constraints.

© 2026 Tertiary Infotech Academy Pte Ltd

How Lab 13 Runs



THE POINT

Frame a business problem as a fine-tuning problem and justify the adaptation method

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 13 · VERIFY

Frame the Problem and Select the Adaptation Method

✅ Expected result

All three options produce output on the same ticket, the option comparison table is printed, CLIP scores the correct caption highest, and adaptation_decision.json records the chosen method with an explicit justification against the stated constraints.

IF IT DOESN'T WORK

CUDA out of memory

Restart the runtime, lower per_device_train_batch_size, raise gradient_accumulation_steps.

No GPU attached

Runtime > Change runtime type > T4 GPU, then re-run from the first cell.

ModuleNotFoundError

Re-run the pip install cell; on Colab, restart the session after installing unsloth.

© 2026 Tertiary Infotech Academy Pte Ltd

LU3 · HANDS-ON · SELF-PACED

Build LoRA from First Principles

LAB 14

Implement the LoRA low-rank decomposition and verify its parameter and memory savings

Implement LoRA yourself in about thirty lines of PyTorch — the low-rank update h = Wx + (alpha/r)BAx — then prove the three properties that make it work: zero-initialised B means an identity start, the parameter count collapses with r, and the merged weight is mathematically identical to the adapted forward pass.

YOU'LL BUILD

A working LoRALinear layer, a parameter-count table across ranks, and a numerical proof that merging BA into W reproduces the adapted output exactly.

TOOLCHAIN

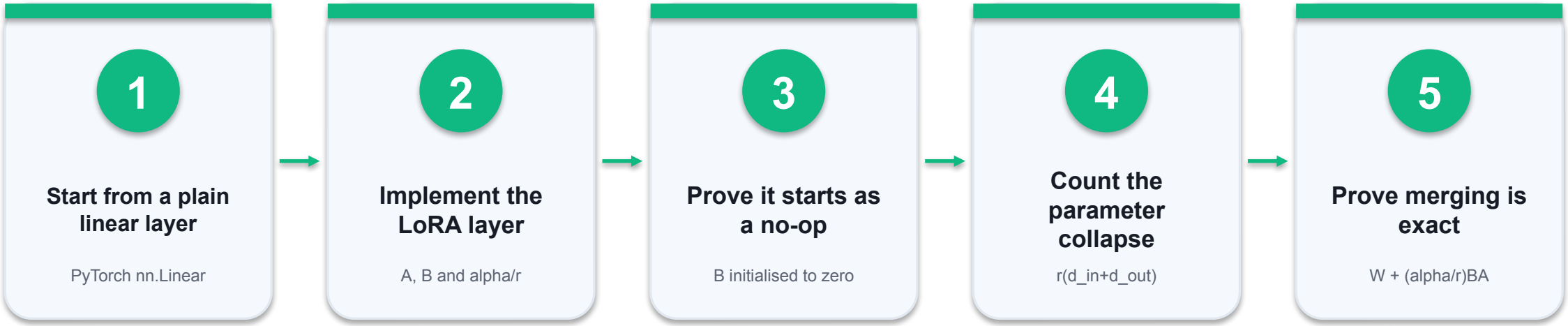
PyTorch, numpy, matplotlib

DONE WHEN

The identity-at-init assertion passes (difference < 1e-6), the parameter table shows LoRA at well under 5% of full parameters for r<=16, the merge assertion passes (error < 1e-4), and lora_param_scaling.png is produced.

© 2026 Tertiary Infotech Academy Pte Ltd

How Lab 14 Runs



THE POINT

Implement the LoRA low-rank decomposition and verify its parameter and memory savings

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 14 · VERIFY

Build LoRA from First Principles

✅ Expected result

The identity-at-init assertion passes (difference < 1e-6), the parameter table shows LoRA at well under 5% of full parameters for r<=16, the merge assertion passes (error < 1e-4), and lora_param_scaling.png is produced.

IF IT DOESN'T WORK

CUDA out of memory

Restart the runtime, lower per_device_train_batch_size, raise gradient_accumulation_steps.

No GPU attached

Runtime > Change runtime type > T4 GPU, then re-run from the first cell.

ModuleNotFoundError

Re-run the pip install cell; on Colab, restart the session after installing unsloth.

© 2026 Tertiary Infotech Academy Pte Ltd

LU3 · HANDS-ON · IN CLASS

Fine-Tune with PEFT LoRA

LAB 15

Fine-tune a pre-trained model with the PEFT LoRA implementation

Run a real LoRA fine-tune with the Hugging Face PEFT library on the dataset built in LU1 and LU2: configure LoraConfig, inspect the trainable parameter count, train with SFTTrainer, and compare the base and adapted model on held-out prompts.

YOU'LL BUILD

A trained LoRA adapter of a few tens of megabytes, a training-loss curve, and a side-by-side before/after comparison on held-out prompts.

TOOLCHAIN

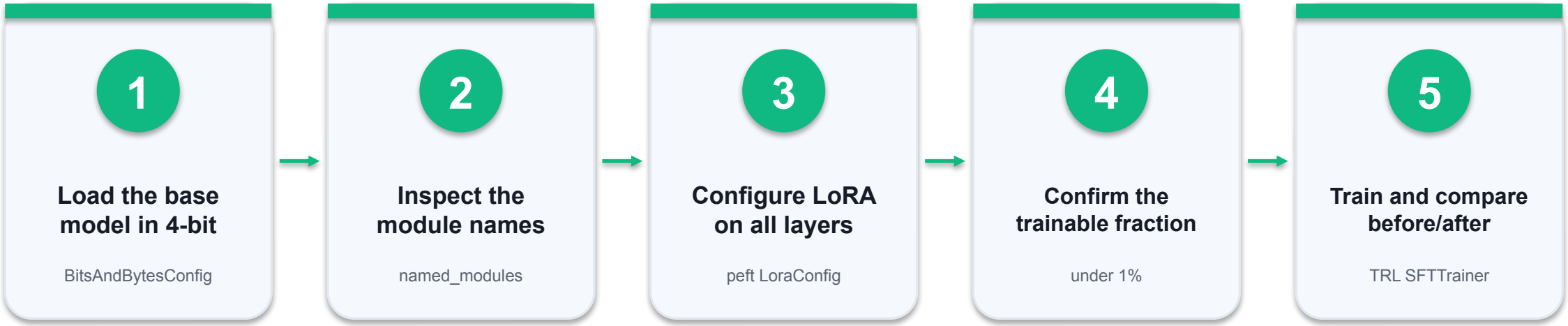
Hugging Face PEFT, transformers, TRL SFTTrainer, bitsandbytes, datasets

DONE WHEN

print_trainable_parameters() reports under 1% trainable, the training loss falls from its starting value, peft_loss_curve.png shows a descending curve, the after-generation differs visibly from the before, and lora_adapter/ is only tens of megabytes.

© 2026 Tertiary Infotech Academy Pte Ltd

How Lab 15 Runs



THE POINT

Fine-tune a pre-trained model with the PEFT LoRA implementation

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 15 · VERIFY

Fine-Tune with PEFT LoRA

✅ Expected result

print_trainable_parameters() reports under 1% trainable, the training loss falls from its starting value, peft_loss_curve.png shows a descending curve, the after-generation differs visibly from the before, and lora_adapter/ is only tens of megabytes.

IF IT DOESN'T WORK

CUDA out of memory

Restart the runtime, lower per_device_train_batch_size, raise gradient_accumulation_steps.

No GPU attached

Runtime > Change runtime type > T4 GPU, then re-run from the first cell.

ModuleNotFoundError

Re-run the pip install cell; on Colab, restart the session after installing unsloth.

© 2026 Tertiary Infotech Academy Pte Ltd

LU3 · HANDS-ON · SELF-PACED

Fine-Tune with QLoRA in 4-Bit

LAB 16

Apply QLoRA 4-bit quantisation and measure the memory saving against LoRA

Quantify what QLoRA actually buys. You measure the VRAM of the same model at 16-bit, 8-bit and 4-bit NF4, inspect the effect of double quantisation and paged optimisers, fine-tune in 4-bit, and compare the resulting quality against the 16-bit LoRA run.

YOU'LL BUILD

A memory-comparison table across three precisions with measured VRAM, a QLoRA fine-tune that fits in a free Colab session, and a quality comparison against the LoRA baseline.

TOOLCHAIN

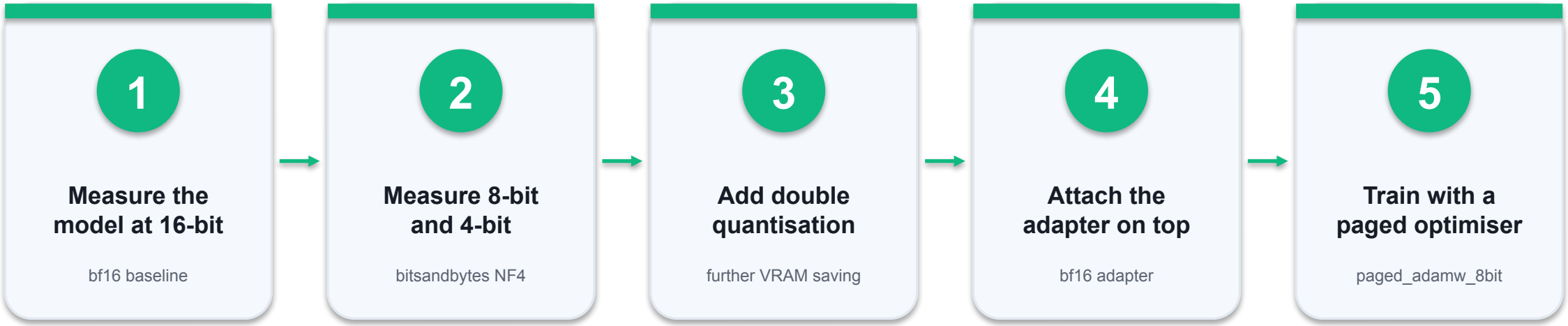
bitsandbytes, PEFT, transformers, TRL, torch

DONE WHEN

quantisation_memory.csv shows 4-bit using substantially less VRAM than 16-bit, the parameter dump shows Params4bit storage with a bf16 adapter, the QLoRA training completes with peak VRAM reported, and qlora_adapter/ is saved.

© 2026 Tertiary Infotech Academy Pte Ltd

How Lab 16 Runs



THE POINT

Apply QLoRA 4-bit quantisation and measure the memory saving against LoRA

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 16 · VERIFY

Fine-Tune with QLoRA in 4-Bit

✅ Expected result

quantisation_memory.csv shows 4-bit using substantially less VRAM than 16-bit, the parameter dump shows Params4bit storage with a bf16 adapter, the QLoRA training completes with peak VRAM reported, and qlora_adapter/ is saved.

IF IT DOESN'T WORK

CUDA out of memory

Restart the runtime, lower per_device_train_batch_size, raise gradient_accumulation_steps.

No GPU attached

Runtime > Change runtime type > T4 GPU, then re-run from the first cell.

ModuleNotFoundError

Re-run the pip install cell; on Colab, restart the session after installing unsloth.

© 2026 Tertiary Infotech Academy Pte Ltd

LU3 · HANDS-ON · IN CLASS

Accelerate Fine Tuning with Unsloth

LAB 17

Use Unsloth to fine-tune faster and with less memory, and verify the speed-up

Run the same fine-tune through Unsloth's optimised kernels and measure the difference against the plain PEFT run: training time, peak VRAM and final loss. You then use the Unsloth chat templates and response-only training that the standard loop makes awkward.

YOU'LL BUILD

A benchmark table comparing Unsloth against standard PEFT on time, memory and loss, plus a trained adapter produced by the Unsloth pipeline.

TOOLCHAIN

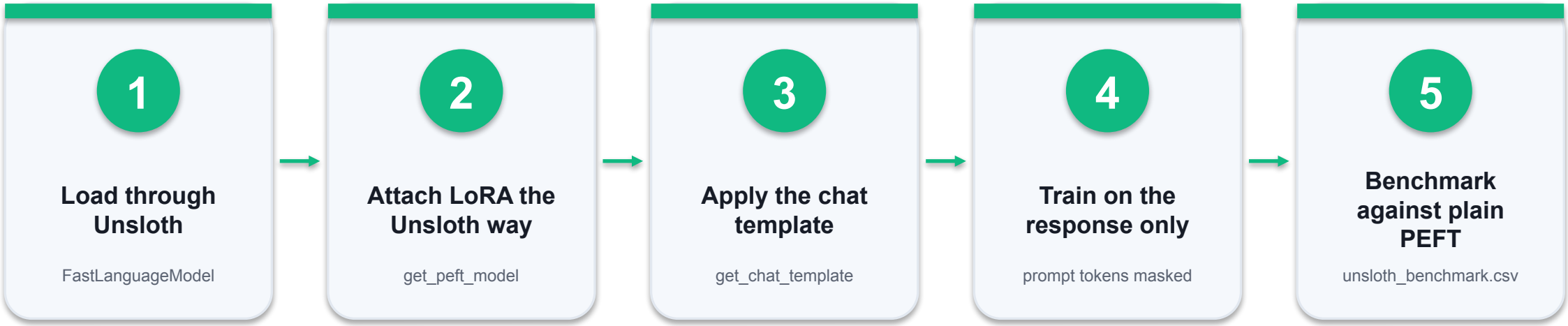
Unsloth FastLanguageModel, TRL SFTTrainer, Triton kernels, datasets

DONE WHEN

The training completes and reports runtime, peak reserved VRAM and the memory used for training; the response-only masking reports a non-zero count of masked prompt tokens; unsloth_benchmark.csv compares Unsloth against the PEFT run; and unsloth_adapter/ is saved.

© 2026 Tertiary Infotech Academy Pte Ltd

How Lab 17 Runs



THE POINT

Use Unsloth to fine-tune faster and with less memory, and verify the speed-up

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 17 · VERIFY

Accelerate Fine Tuning with Unsloth

✅ Expected result

The training completes and reports runtime, peak reserved VRAM and the memory used for training; the response-only masking reports a non-zero count of masked prompt tokens; unsloth_benchmark.csv compares Unsloth against the PEFT run; and unsloth_adapter/ is saved.

IF IT DOESN'T WORK

CUDA out of memory

Restart the runtime, lower per_device_train_batch_size, raise gradient_accumulation_steps.

No GPU attached

Runtime > Change runtime type > T4 GPU, then re-run from the first cell.

ModuleNotFoundError

Re-run the pip install cell; on Colab, restart the session after installing unsloth.

© 2026 Tertiary Infotech Academy Pte Ltd

LU3 · HANDS-ON · SELF-PACED

Monitor Training Stability and Convergence

LAB 18

Monitor a generative training run for stability and convergence, and correct failures

Learn to read a training run. You instrument loss, gradient norm, learning rate and VRAM with a custom callback, then deliberately induce the three classic failures — divergence, overfitting and a stalled run — recognise each signature and apply the correct fix.

YOU'LL BUILD

A monitoring callback producing a four-panel diagnostic dashboard, plus three reproduced failure modes each with its identified signature and applied correction.

TOOLCHAIN

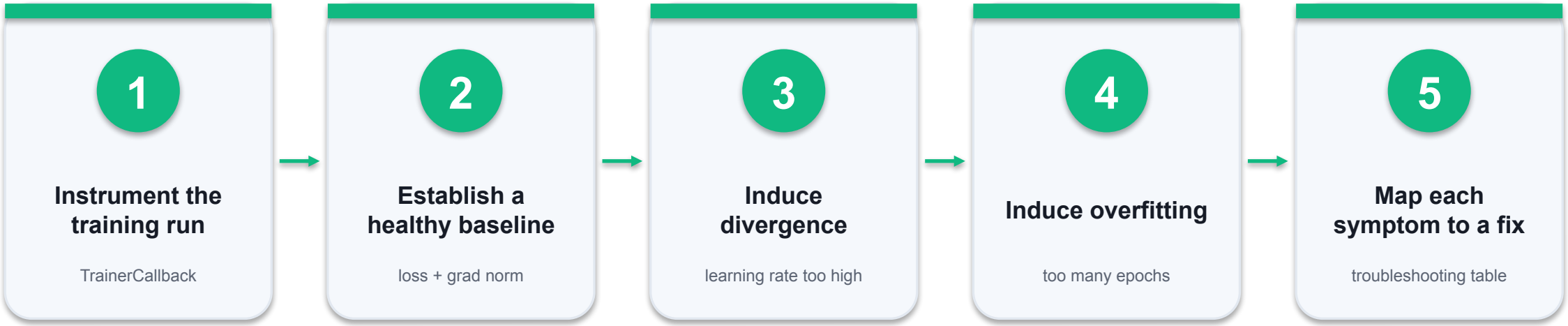
TRL SFTTrainer, transformers TrainerCallback, matplotlib, torch

DONE WHEN

monitor_healthy.png, monitor_diverged.png, monitor_overfit.png and monitor_stalled.png are all produced; diagnose() correctly labels the healthy run HEALTHY and each broken run with its specific failure; and training_troubleshooting.csv maps every symptom to a fix.

© 2026 Tertiary Infotech Academy Pte Ltd

How Lab 18 Runs



THE POINT

Monitor a generative training run for stability and convergence, and correct failures

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 18 · VERIFY

Monitor Training Stability and Convergence

✅ Expected result

monitor_healthy.png, monitor_diverged.png, monitor_overfit.png and monitor_stalled.png are all produced; diagnose() correctly labels the healthy run HEALTHY and each broken run with its specific failure; and training_troubleshooting.csv maps every symptom to a fix.

IF IT DOESN'T WORK

CUDA out of memory

Restart the runtime, lower per_device_train_batch_size, raise gradient_accumulation_steps.

No GPU attached

Runtime > Change runtime type > T4 GPU, then re-run from the first cell.

ModuleNotFoundError

Re-run the pip install cell; on Colab, restart the session after installing unsloth.

© 2026 Tertiary Infotech Academy Pte Ltd

LU3 · HANDS-ON · SELF-PACED

Hyperparameter Optimization for LoRA

LAB 19

Perform hyperparameter optimization and select a configuration on measured evidence

Run a systematic hyperparameter search in the correct priority order — learning rate, then rank and alpha, then target modules, then epochs and effective batch size — measuring validation loss for each configuration and selecting the winner on evidence.

YOU'LL BUILD

A hyperparameter sweep table with validation loss per configuration, sensitivity plots, and a selected configuration saved as best_hyperparameters.json.

TOOLCHAIN

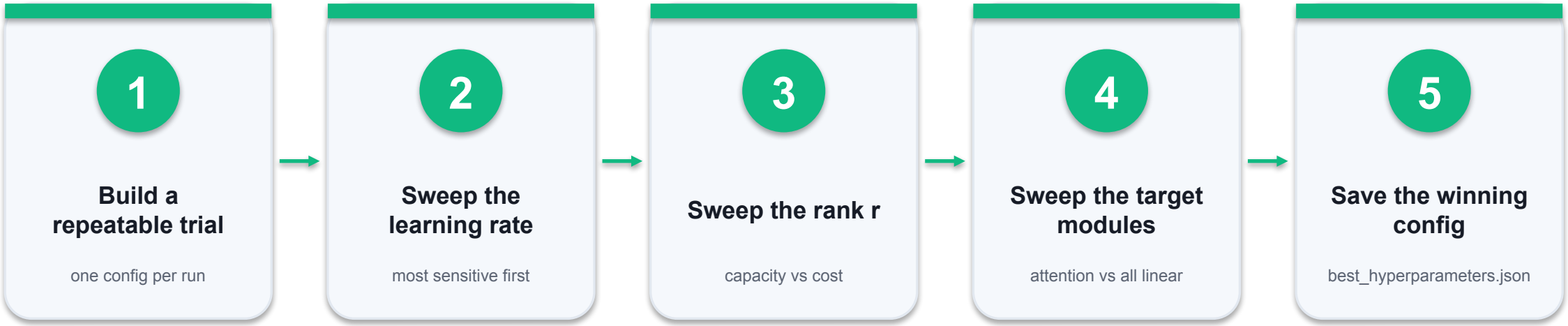
Unsloth, TRL SFTTrainer, pandas, matplotlib, numpy

DONE WHEN

hpo_sweep.csv records every trial with its validation loss; hpo_sensitivity.png shows all four sensitivity curves; the target-module comparison shows 'all' beating 'attn_only'; and best_hyperparameters.json holds the selected configuration.

© 2026 Tertiary Infotech Academy Pte Ltd

How Lab 19 Runs



THE POINT

Perform hyperparameter optimization and select a configuration on measured evidence

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 19 · VERIFY

Hyperparameter Optimization for LoRA

✅ Expected result

hpo_sweep.csv records every trial with its validation loss; hpo_sensitivity.png shows all four sensitivity curves; the target-module comparison shows 'all' beating 'attn_only'; and best_hyperparameters.json holds the selected configuration.

IF IT DOESN'T WORK

CUDA out of memory

Restart the runtime, lower per_device_train_batch_size, raise gradient_accumulation_steps.

No GPU attached

Runtime > Change runtime type > T4 GPU, then re-run from the first cell.

ModuleNotFoundError

Re-run the pip install cell; on Colab, restart the session after installing unsloth.

© 2026 Tertiary Infotech Academy Pte Ltd

LU3 · HANDS-ON · SELF-PACED

Advanced Loss Functions and Generative Metrics

LAB 20

Apply advanced loss functions and metrics for evaluating generative performance

Implement and interpret the loss functions and metrics that generative models are actually judged on: cross-entropy and perplexity, BLEU/ROUGE/BERTScore for text, FID and Inception Score for images, CLIPScore for text-image alignment, and the contrastive and diffusion objectives.

YOU'LL BUILD

A metric toolkit with each measure implemented and demonstrated on real outputs, plus a selection guide mapping each generative task to the metrics that are valid for it.

TOOLCHAIN

PyTorch, Hugging Face evaluate, torchmetrics, CLIP, scipy, numpy

DONE WHEN

Perplexity is lower for well-formed than for shuffled text; BERTScore rates the paraphrase higher than BLEU does; FID is lower for the near-real images than the unrelated ones; CLIPScore is higher for the correct caption; and metric_selection_guide.csv is written.

© 2026 Tertiary Infotech Academy Pte Ltd

How Lab 20 Runs

1. **Measure cross-entropy and PPL**
   language modelling loss
2. **Score BLEU and ROUGE**
   evaluate library
3. **Score BERTScore semantics**
   recognises paraphrase
4. **Compute FID for images**
   torchmetrics
5. **Score text-image alignment**
   CLIPScore

THE POINT

Apply advanced loss functions and metrics for evaluating generative performance

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 20 · VERIFY

Advanced Loss Functions and Generative Metrics

✅ Expected result

Perplexity is lower for well-formed than for shuffled text; BERTScore rates the paraphrase higher than BLEU does; FID is lower for the near-real images than the unrelated ones; CLIPScore is higher for the correct caption; and metric_selection_guide.csv is written.

IF IT DOESN'T WORK

CUDA out of memory

Restart the runtime, lower per_device_train_batch_size, raise gradient_accumulation_steps.

No GPU attached

Runtime > Change runtime type > T4 GPU, then re-run from the first cell.

ModuleNotFoundError

Re-run the pip install cell; on Colab, restart the session after installing unsloth.

© 2026 Tertiary Infotech Academy Pte Ltd

Recap — Modelling

1

Lab 13

Frame a business problem as a fine-tuning problem and justify the adaptation method

2

Lab 14

Implement the LoRA low-rank decomposition and verify its parameter and memory savings

3

Lab 15

Fine-tune a pre-trained model with the PEFT LoRA implementation

4

Lab 16

Apply QLoRA 4-bit quantisation and measure the memory saving against LoRA

5

Lab 17

Use Unsloth to fine-tune faster and with less memory, and verify the speed-up

6

Lab 18

Monitor a generative training run for stability and convergence, and correct failures

© 2026 Tertiary Infotech Academy Pte Ltd

Common Mistakes — Modelling

1

Targeting attention only

q_proj and v_proj alone underfit badly. Include the MLP gate/up/down projections.

2

Copying a learning rate from a blog

2e-4 suits LoRA on a small set; the right value depends on your data and rank.

3

Too many epochs

On a small dataset this buys overfitting, not quality. Watch the validation curve.

4

Reading one loss number

A single value cannot distinguish healthy convergence from overfitting — read the curve.

5

Raising rank to fix quality

Validation loss usually saturates by r=16. If quality is poor, look at the DATA first.

6

No validation split

Without one, overfitting is invisible until the model reaches production.

© 2026 Tertiary Infotech Academy Pte Ltd

04

LEARNING UNIT 04

Machine Learning Implementation and Operations

Benchmarking against state of the art · explainability and interpretability · bias and fairness · continuous pre-training pipelines · deployment

© 2026 Tertiary Infotech Academy Pte Ltd

LU4 at a Glance

1

Learning outcome

Analyse ML solutions for bias, explainability, and alignment with performance benchmarks.

2

Hands-on labs

Lab 21 in class · Labs 22–26 self-paced

3

Weighting

20% of course time · Late afternoon

4

Topics

Build and benchmark generative solutions against state of the art · Recommend and implement bias mitigation and fairness interventions · Apply interpretability and explainability methods · Deploy and operationalize fine-tuned models

© 2026 Tertiary Infotech Academy Pte Ltd

Key Concepts — Machine Learning Implementation and Operations

1. Benchmarking against state of the art needs a held-out set the model has never seen, a fixed decoding configuration, the same prompt template for every system, and more than one metric.

2. Standard public benchmarks — MMLU for knowledge, GSM8K for reasoning, HumanEval for code, HELM and the Open LLM Leaderboard for breadth — provide comparison points, but a domain-specific held-out set is what actually predicts production performance.

3. LLM-as-a-judge scores open-ended generation on correctness, helpfulness and format at a fraction of human cost, but must be validated against human labels and controlled for position bias.

4. Always benchmark against the base model, not only against the previous fine-tune — a fine-tune that beats its predecessor but loses to the base model is a regression.

© 2026 Tertiary Infotech Academy Pte Ltd

Key Concepts — Machine Learning Implementation and Operations (2)

1. Catastrophic forgetting is measured, not assumed: re-run a general benchmark after fine tuning to confirm the model has not lost its original capability.

2. Interpretability and explainability for generative models: attention visualisation, integrated gradients and SHAP for token attribution, probing classifiers for what a layer encodes, and chain-of-thought as a (partial) behavioural explanation.

3. Attention weights show where the model looked, not why it decided — treat attention maps as a hypothesis-generating tool, not a causal proof.

4. Sources of bias in generative models: the pre-training corpus, the fine-tuning set, annotator judgement, the objective function itself, and deployment feedback loops that amplify whatever the model already prefers.

© 2026 Tertiary Infotech Academy Pte Ltd

Key Concepts — Machine Learning Implementation and Operations (3)

1. Fairness-enhancing interventions act at three stages — pre-processing (rebalance and augment the data), in-processing (counterfactual data augmentation, fairness-regularised or preference objectives), and post-processing (output filters, safety classifiers, guardrails).

2. Counterfactual evaluation swaps demographic attributes in an otherwise identical prompt and measures the change in the output — a large change is direct evidence of bias.

3. Continuous pre-training pipelines resume from an existing checkpoint on new raw domain text, with a low learning rate, a replay mixture of the original data and regular general-benchmark checks to guard against forgetting.

4. Checkpoint hygiene makes a pipeline restartable: save optimizer and scheduler state alongside weights, version every checkpoint against its dataset revision, and validate each cycle before promoting it.

© 2026 Tertiary Infotech Academy Pte Ltd

Key Concepts — Machine Learning Implementation and Operations (4)

1. Deployment: merge to 16-bit for a GPU server (vLLM), export GGUF q4_k_m for CPU/local serving (llama.cpp, Ollama), or keep the adapter separate and hot-swap it per tenant.

2. Model cards and dataset cards document intended use, training data, evaluation results and known limitations — they are the governance artifact that makes a fine-tune auditable.

© 2026 Tertiary Infotech Academy Pte Ltd

Benchmarking That Means Something

1

A held-out set

Data the model has NEVER seen. Benchmarking on training data flatters and misleads.

2

A fixed decoder

Same temperature, top-p and max tokens for every system compared.

3

The same prompt template

A template difference alone can move a score by several points.

4

More than one metric

Every metric is blind to a different failure.

5

The BASE model as control

A fine-tune that loses to its own base model is a regression.

6

A forgetting check

Re-run a general benchmark — measure what the fine-tune cost you elsewhere.

© 2026 Tertiary Infotech Academy Pte Ltd

LU4 · THE CONTROL

Always benchmark against the base model, not only the previous fine-tune.

A model that beats its predecessor but loses to the untouched base model is a regression that a leaderboard of your own runs will never reveal.

© 2026 Tertiary Infotech Academy Pte Ltd

Interpretability Methods and Their Limits

1

Attention visualisation

Shows where each head LOOKED. Correlational — a high weight is not proof of influence.

2

Integrated gradients

Causal attribution to input tokens. Depends on the baseline; expensive on long text.

3

Probing classifiers

Finds WHERE a property becomes decodable. Decodable does not mean the model uses it.

4

Behavioural ablation

Remove a token, measure the change. Ignores interactions between tokens.

5

Chain of thought

A stated reason — but not necessarily the actual computation.

6

Model cards

The governance artifact: intended use, data, evaluation and known limitations.

© 2026 Tertiary Infotech Academy Pte Ltd

LU4 · THE LIMIT

Attention shows where the model looked. It does not show why it decided.

Use at least two methods before making a claim to a regulator. An explanation supported only by an attention map is not defensible.

© 2026 Tertiary Infotech Academy Pte Ltd

Five Sources of Bias in Generative Models

1

Pre-training corpus

Web-scale text carries historical and demographic skew into every downstream model.

2

Fine-tuning dataset

Whatever you chose to include — and whatever you failed to include.

3

Annotator judgement

Who wrote the reference answers, and what did they consider a good answer?

4

Objective function

Maximum likelihood optimises for the majority pattern, not for fairness.

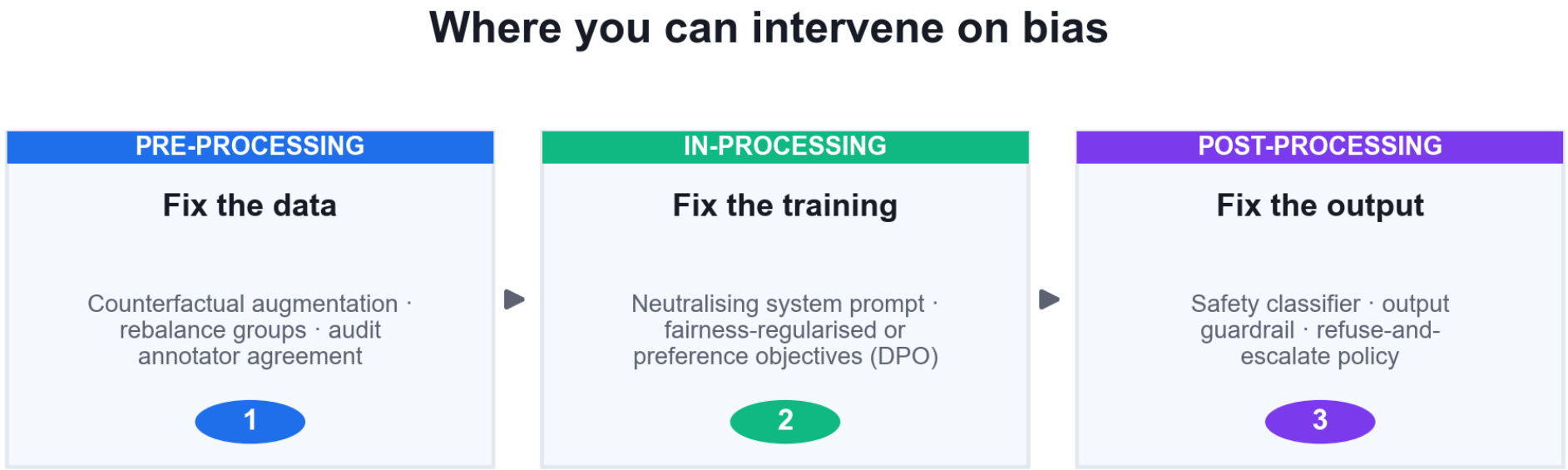
5

Deployment feedback loop

The model's own outputs become tomorrow's training data, amplifying its preferences.

© 2026 Tertiary Infotech Academy Pte Ltd

Where You Can Intervene on Bias



Counterfactual testing before and after every intervention is what turns a claim into a mitigation.

© 2026 Tertiary Infotech Academy Pte Ltd

A Continuous Pre-Training Cycle

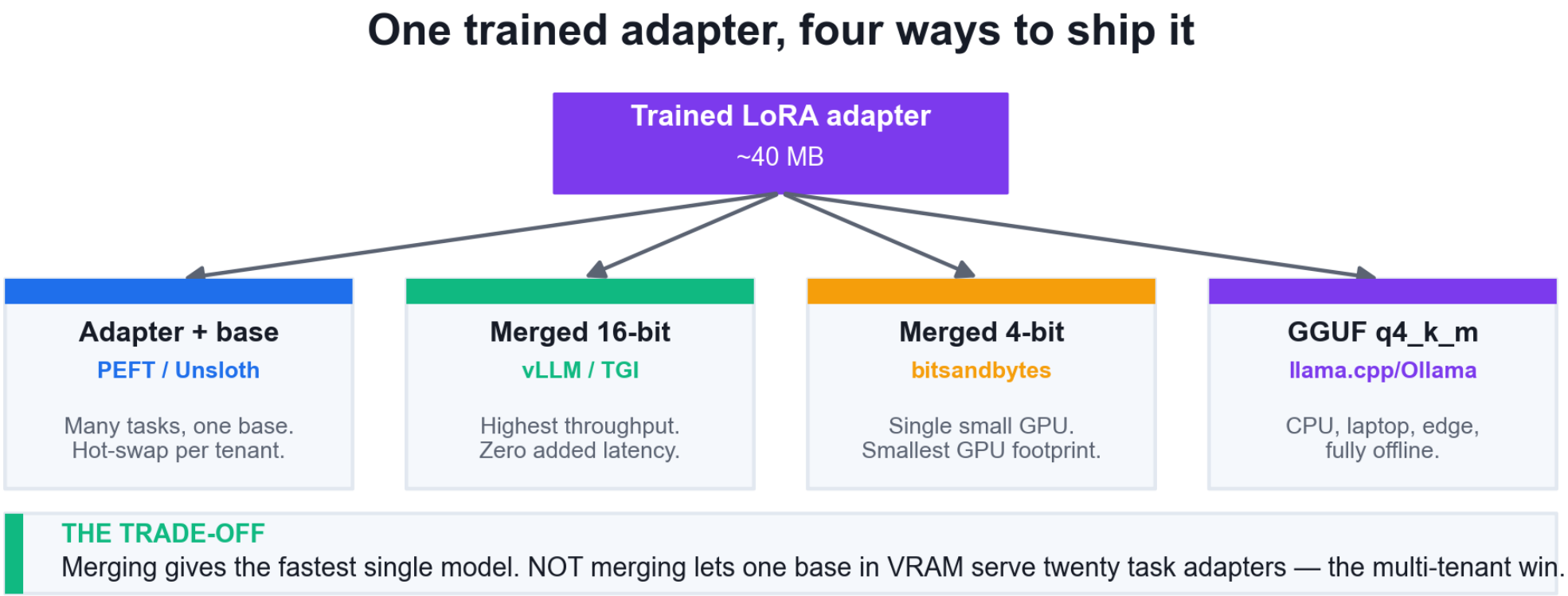


WHAT MAKES IT RESTARTABLE

Saving optimiser and scheduler state alongside the weights, and versioning every checkpoint against its dataset revision.

© 2026 Tertiary Infotech Academy Pte Ltd

One Adapter, Four Ways to Ship It



Merging gives the fastest single model. NOT merging lets one base in VRAM serve twenty task adapters — the multi-tenant win.

© 2026 Tertiary Infotech Academy Pte Ltd

WHAT YOU'LL DO

Hands-On Labs — Machine Learning Implementation and Operations

Labs 21–22

- Benchmark the Fine-Tuned Model Against State of the Art
- Detect and Mitigate Bias in Generative Outputs

Labs 23–24

- Interpretability and Explainability for Generative Models
- Intermediate Fine Tuning and Continuous Pre-Training

Labs 25–26

- Merge, Export and Deploy the Fine-Tuned Model
- Capstone — End-to-End Fine-Tuning Project

© 2026 Tertiary Infotech Academy Pte Ltd

LU4 · HANDS-ON · IN CLASS

Benchmark the Fine-Tuned Model Against State of the Art

LAB 21

Analyse a fine-tuned model against state-of-the-art benchmarks

Run a rigorous benchmark: base model versus fine-tuned model versus a reference model on a held-out set with a fixed decoding configuration, using several metrics plus an LLM judge, and check explicitly for catastrophic forgetting on a general benchmark.

YOU'LL BUILD

A benchmark report comparing three systems on four metrics with a catastrophic-forgetting check, and a recommendation grounded in the measured gap.

TOOLCHAIN

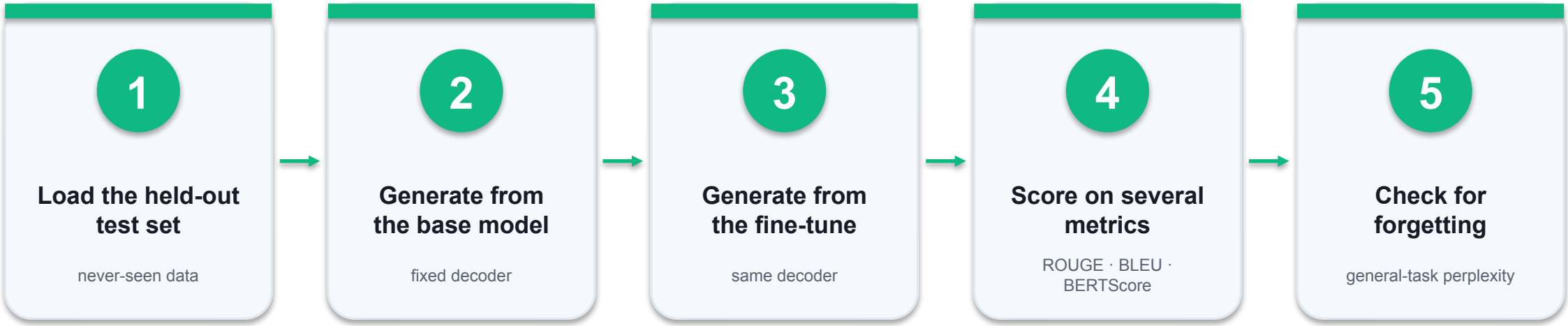
Unsloth, PEFT, Hugging Face evaluate, pandas, matplotlib

DONE WHEN

benchmark_report.json compares base and fine-tuned on ROUGE, BLEU, BERTScore and perplexity under one fixed decoding config; the catastrophic-forgetting check reports a percentage change on general text; and benchmark_comparison.png is produced.

© 2026 Tertiary Infotech Academy Pte Ltd

How Lab 21 Runs



THE POINT

Analyse a fine-tuned model against state-of-the-art benchmarks

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 21 · VERIFY

Benchmark the Fine-Tuned Model Against State of the Art

✅ Expected result

benchmark_report.json compares base and fine-tuned on ROUGE, BLEU, BERTScore and perplexity under one fixed decoding config; the catastrophic-forgetting check reports a percentage change on general text; and benchmark_comparison.png is produced.

IF IT DOESN'T WORK

CUDA out of memory

Restart the runtime, lower per_device_train_batch_size, raise gradient_accumulation_steps.

No GPU attached

Runtime > Change runtime type > T4 GPU, then re-run from the first cell.

ModuleNotFoundError

Re-run the pip install cell; on Colab, restart the session after installing unsloth.

© 2026 Tertiary Infotech Academy Pte Ltd

LU4 · HANDS-ON · SELF-PACED

Detect and Mitigate Bias in Generative Outputs

LAB 22

Identify sources of bias and implement fairness-enhancing interventions

Audit a fine-tuned model for demographic bias with counterfactual testing, trace the bias back to its source in the training data, then implement and measure interventions at all three stages — pre-processing, in-processing and post-processing.

YOU'LL BUILD

A counterfactual bias audit with measured disparity before and after intervention, a data-side rebalancing, an output guardrail, and a fairness report.

TOOLCHAIN

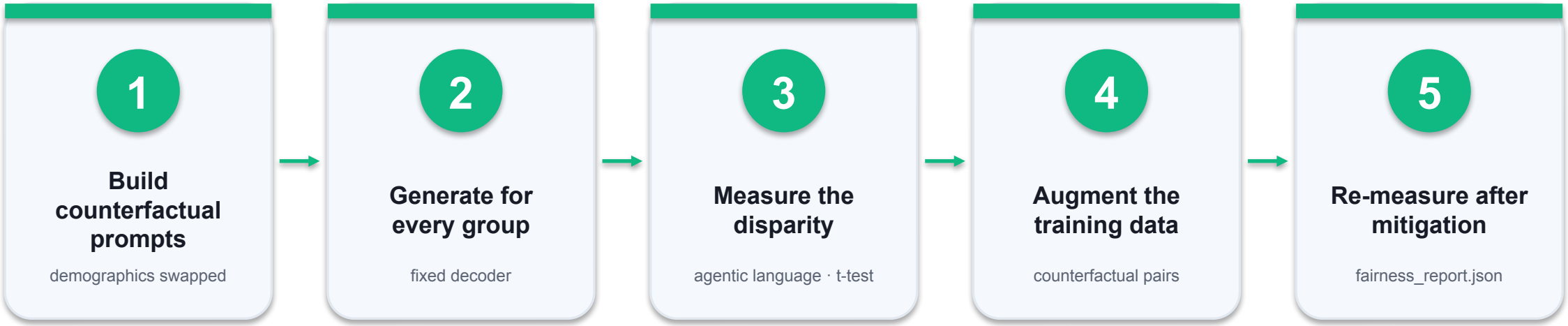
Unsloth, sentence-transformers, pandas, scipy, matplotlib

DONE WHEN

The counterfactual audit reports an agentic-language gap and a length t-test p-value before mitigation; debiased_train.jsonl is larger than the original; the guardrail blocks the biased probe and passes the neutral one; bias_mitigation.png and fairness_report.json are produced.

© 2026 Tertiary Infotech Academy Pte Ltd

How Lab 22 Runs



THE POINT

Identify sources of bias and implement fairness-enhancing interventions

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 22 · VERIFY

Detect and Mitigate Bias in Generative Outputs

✅ Expected result

The counterfactual audit reports an agentic-language gap and a length t-test p-value before mitigation; debiased_train.jsonl is larger than the original; the guardrail blocks the biased probe and passes the neutral one; bias_mitigation.png and fairness_report.json are produced.

IF IT DOESN'T WORK

CUDA out of memory

Restart the runtime, lower per_device_train_batch_size, raise gradient_accumulation_steps.

No GPU attached

Runtime > Change runtime type > T4 GPU, then re-run from the first cell.

ModuleNotFoundError

Re-run the pip install cell; on Colab, restart the session after installing unsloth.

© 2026 Tertiary Infotech Academy Pte Ltd

LU4 · HANDS-ON · SELF-PACED

Interpretability and Explainability for Generative Models

LAB 23

Apply interpretability and explainability methods to a generative model

Open the model up with four complementary methods — attention visualisation, gradient-based token attribution, a layer probing classifier and behavioural ablation — and understand what each one can and cannot tell a regulator.

YOU'LL BUILD

Four working explainability analyses with plots, plus a written note on the limits of each method — the kind of evidence a compliance officer actually asks for.

TOOLCHAIN

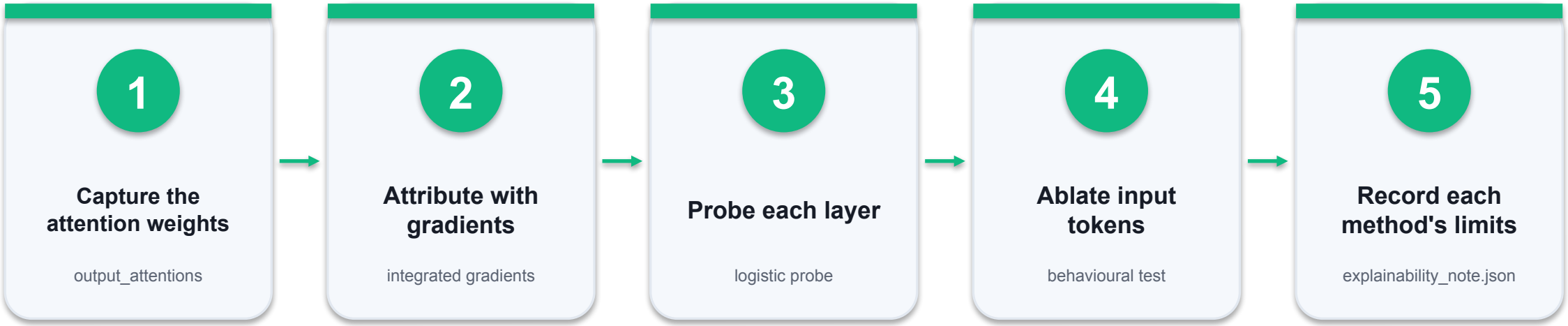
PyTorch, transformers, matplotlib, seaborn, scikit-learn

DONE WHEN

explain_attention.png, explain_attention_vs_ig.png and explain_probing.png are produced; the attention-vs-IG correlation is reported; the probing curve identifies a peak layer above chance; the ablation table ranks the tokens; and explainability_note.json records each method's limits.

© 2026 Tertiary Infotech Academy Pte Ltd

How Lab 23 Runs



THE POINT

Apply interpretability and explainability methods to a generative model

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 23 · VERIFY

Interpretability and Explainability for Generative Models

✅ Expected result

explain_attention.png, explain_attention_vs_ig.png and explain_probing.png are produced; the attention-vs-IG correlation is reported; the probing curve identifies a peak layer above chance; the ablation table ranks the tokens; and explainability_note.json records each method's limits.

IF IT DOESN'T WORK

CUDA out of memory

Restart the runtime, lower per_device_train_batch_size, raise gradient_accumulation_steps.

No GPU attached

Runtime > Change runtime type > T4 GPU, then re-run from the first cell.

ModuleNotFoundError

Re-run the pip install cell; on Colab, restart the session after installing unsloth.

© 2026 Tertiary Infotech Academy Pte Ltd

LU4 · HANDS-ON · SELF-PACED

Intermediate Fine Tuning and Continuous Pre-Training

LAB 24

Build a training pipeline that continues pre-training from an existing checkpoint

Distinguish the three adaptation strategies in practice: direct task fine tuning, intermediate fine tuning on a related corpus first, and continuous pre-training on raw domain text. You build a restartable continuous pre-training pipeline with replay data and forgetting checks.

YOU'LL BUILD

A checkpoint-resumable continuous pre-training pipeline with a replay mixture, per-cycle validation and a comparison of the three adaptation strategies on the same held-out set.

TOOLCHAIN

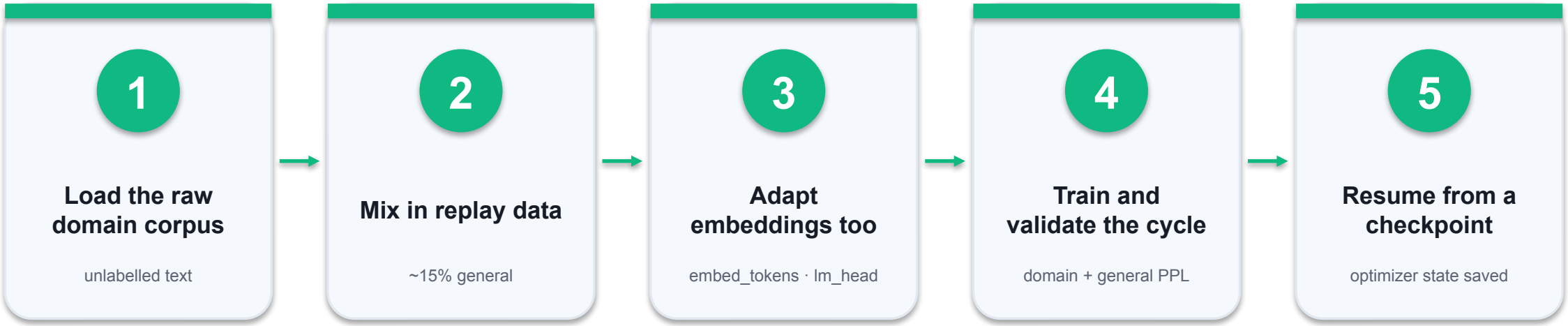
Unsloth, TRL SFTTrainer, transformers, datasets, torch

DONE WHEN

cpt_cycle_log.json records baseline plus two cycles with domain and general perplexity for each; cycle 2 successfully resumes from a saved checkpoint; the replay ratio is reported; and the forgetting guard produces an accept/reject verdict.

© 2026 Tertiary Infotech Academy Pte Ltd

How Lab 24 Runs



THE POINT

Build a training pipeline that continues pre-training from an existing checkpoint

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 24 · VERIFY

Intermediate Fine Tuning and Continuous Pre-Training

✅ Expected result

cpt_cycle_log.json records baseline plus two cycles with domain and general perplexity for each; cycle 2 successfully resumes from a saved checkpoint; the replay ratio is reported; and the forgetting guard produces an accept/reject verdict.

IF IT DOESN'T WORK

CUDA out of memory

Restart the runtime, lower per_device_train_batch_size, raise gradient_accumulation_steps.

No GPU attached

Runtime > Change runtime type > T4 GPU, then re-run from the first cell.

ModuleNotFoundError

Re-run the pip install cell; on Colab, restart the session after installing unsloth.

© 2026 Tertiary Infotech Academy Pte Ltd

LU4 · HANDS-ON · SELF-PACED

Merge, Export and Deploy the Fine-Tuned Model

LAB 25

Deploy and operationalize a fine-tuned model across serving targets

Take the adapter to production: merge it into the base weights, verify the merge changed nothing it should not, export to 16-bit and GGUF, benchmark merged versus adapter-only serving, and write the model card that makes the deployment auditable.

YOU'LL BUILD

A merged 16-bit model, a GGUF quantised export, a latency and memory comparison of the serving options, and a completed model card.

TOOLCHAIN

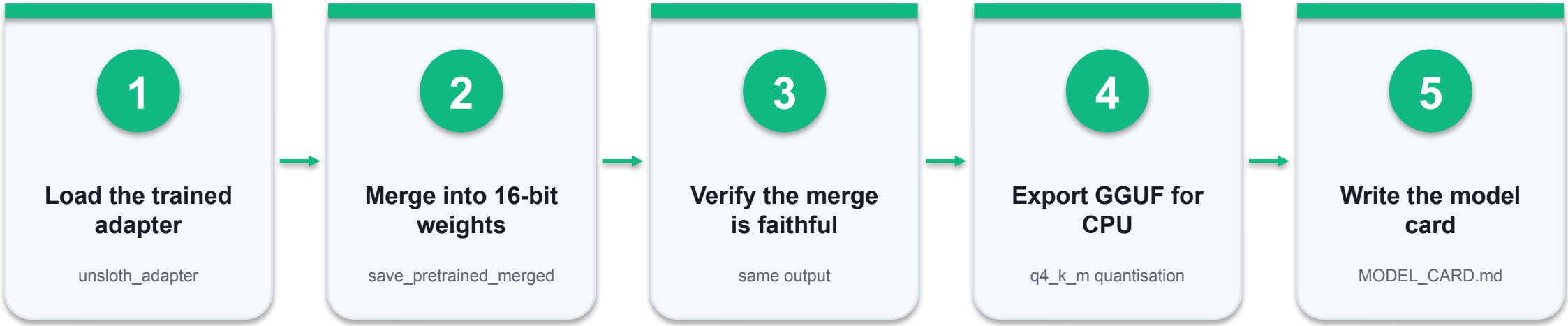
Unsloth, PEFT, llama.cpp GGUF, vLLM, Hugging Face Hub

DONE WHEN

merged_16bit/, merged_4bit/ and gguf_out/ all exist with a .gguf file present; the merged model produces output comparable to the adapter version; deployment_options.csv compares all four serving options by size; and MODEL_CARD.md is written with real measured numbers.

© 2026 Tertiary Infotech Academy Pte Ltd

How Lab 25 Runs



THE POINT

Deploy and operationalize a fine-tuned model across serving targets

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 25 · VERIFY

Merge, Export and Deploy the Fine-Tuned Model

✅ Expected result

merged_16bit/, merged_4bit/ and gguf_out/ all exist with a .gguf file present; the merged model produces output comparable to the adapter version; deployment_options.csv compares all four serving options by size; and MODEL_CARD.md is written with real measured numbers.

IF IT DOESN'T WORK

CUDA out of memory

Restart the runtime, lower per_device_train_batch_size, raise gradient_accumulation_steps.

No GPU attached

Runtime > Change runtime type > T4 GPU, then re-run from the first cell.

ModuleNotFoundError

Re-run the pip install cell; on Colab, restart the session after installing unsloth.

© 2026 Tertiary Infotech Academy Pte Ltd

LU4 · HANDS-ON · SELF-PACED

Capstone — End-to-End Fine-Tuning Project

LAB 26

Deliver a complete fine-tuning solution from data to deployment with governance

Put the whole course together on a new domain: build the dataset, fine-tune with a justified configuration, benchmark against the base model, audit for bias, and deliver the model card and deployment artifacts as one reviewable package.

YOU'LL BUILD

A complete, reviewable fine-tuning project: versioned dataset, trained adapter, benchmark report, fairness audit, model card and deployment export.

TOOLCHAIN

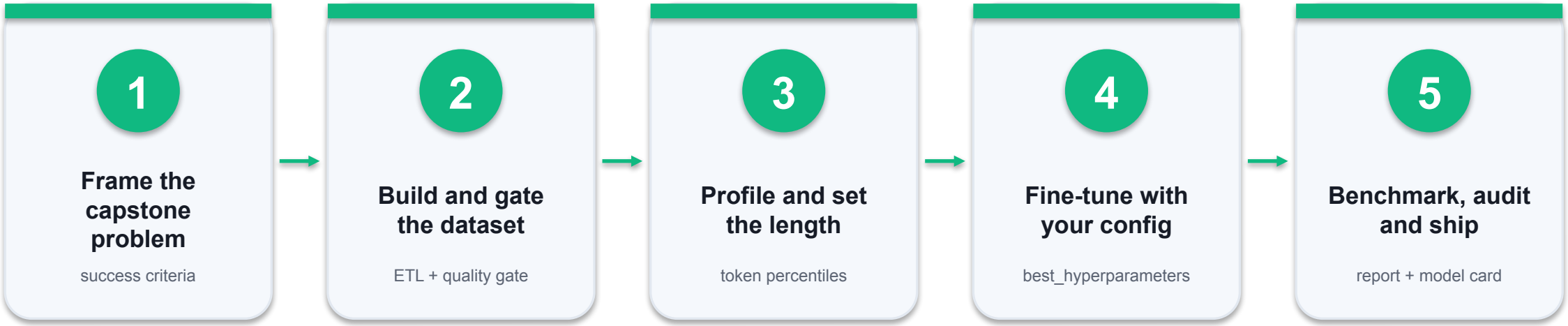
The full course stack — datasets, Unsloth, PEFT, TRL, evaluate, matplotlib

DONE WHEN

capstone/PROJECT_REPORT.json records the dataset sizes, the justified method, the training metrics, the benchmark scores and the fairness spread; capstone/adapter/ and capstone/merged_16bit/ both exist; and the quality-gate assertion passed.

© 2026 Tertiary Infotech Academy Pte Ltd

How Lab 26 Runs



THE POINT

Deliver a complete fine-tuning solution from data to deployment with governance

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 26 · VERIFY

Capstone — End-to-End Fine-Tuning Project

✅ Expected result

capstone/PROJECT_REPORT.json records the dataset sizes, the justified method, the training metrics, the benchmark scores and the fairness spread; capstone/adapter/ and capstone/merged_16bit/ both exist; and the quality-gate assertion passed.

IF IT DOESN'T WORK

CUDA out of memory

Restart the runtime, lower per_device_train_batch_size, raise gradient_accumulation_steps.

No GPU attached

Runtime > Change runtime type > T4 GPU, then re-run from the first cell.

ModuleNotFoundError

Re-run the pip install cell; on Colab, restart the session after installing unsloth.

© 2026 Tertiary Infotech Academy Pte Ltd

Recap — Machine Learning Implementation and Operations

1

Lab 21

Analyse a fine-tuned model against state-of-the-art benchmarks

2

Lab 22

Identify sources of bias and implement fairness-enhancing interventions

3

Lab 23

Apply interpretability and explainability methods to a generative model

4

Lab 24

Build a training pipeline that continues pre-training from an existing checkpoint

5

Lab 25

Deploy and operationalize a fine-tuned model across serving targets

6

Lab 26

Deliver a complete fine-tuning solution from data to deployment with governance

© 2026 Tertiary Infotech Academy Pte Ltd

Common Mistakes — Machine Learning Implementation and Operations

1

Benchmarking on training data

It produces a flattering number that predicts nothing.

2

Only comparing to the previous fine-tune

A model that beats its predecessor but loses to the base model is a regression.

3

Skipping the forgetting check

Domain gains routinely cost general capability. Measure it on a task you did not target.

4

Claiming fairness without measurement

An unmeasured intervention is a claim, not a mitigation. Run a counterfactual audit.

5

Explaining with attention alone

Attention is correlational. Support any regulatory claim with a second, causal method.

6

Shipping without a model card

Intended use, data, evaluation and limitations — without these the deployment is not auditable.

© 2026 Tertiary Infotech Academy Pte Ltd

WRAP-UP

Course Summary & Next Steps

© 2026 Tertiary Infotech Academy Pte Ltd

LEARNING OUTCOMES

What You Achieved

1

LO1 · Ingested and prepared data

Built a versioned data repository, generated synthetic data, ingested from four sources and transformed to the model's chat template.

2

LO2 · Optimised data pipelines

Sanitised a corpus end to end, engineered prompt-side features, built an ETL pipeline with a quality gate, and profiled the dataset.

3

LO3 · Fine-tuned models

Implemented LoRA from first principles, then fine-tuned with PEFT, QLoRA and Unsloth, monitored stability and optimised hyperparameters.

4

LO4 · Analysed for bias and benchmarks

Benchmarked against the base model, audited for demographic bias, applied explainability methods and deployed the result.

© 2026 Tertiary Infotech Academy Pte Ltd

THE TAKEAWAYS

The Ten Things to Remember

1. **Diagnose before you train**
   Prompt engineering fixes instructions, RAG fixes facts, fine tuning fixes behaviour.

2. **B starts at zero**
   LoRA begins as an exact no-op and only ever adds to the frozen base.

3. **Target ALL linear layers**
   Attention-only targeting is the most common cause of a disappointing fine-tune.

4. **Clean beats big**
   A thousand curated examples beat a hundred thousand dirty ones.

5. **Get the chat template right**
   A template mismatch trains cleanly and behaves badly. Always append EOS.

6. **Learning rate first**
   2e-4 for LoRA. Tune it before anything else; it dominates every other choice.

7. **Read the curve, not the number**
   Divergence, overfitting and stalling each have a distinct signature.

8. **Benchmark against the base**
   And check for catastrophic forgetting on a task you did not target.

9. **Measure bias, don't assume it**
   Counterfactual testing before and after every intervention.

10. **Ship the model card**
    Intended use, data, evaluation and limitations — or the deployment is not auditable.

© 2026 Tertiary Infotech Academy Pte Ltd

NEXT STEPS

Continue Your Learning

1

Unsloth documentation

https://unsloth.ai/docs — the reference for every FastLanguageModel option.

2

Hugging Face PEFT

https://huggingface.co/docs/peft — LoRA, DoRA, AdaLoRA and the full adapter family.

3

Hugging Face TRL

https://huggingface.co/docs/trl — SFTTrainer, DPO and preference optimisation.

4

Redo the labs

Rebuild each lab from a blank notebook until the workflow is automatic.

5

Fine-tune your own data

Take the capstone pipeline and point it at a dataset from your own work.

6

Publish it

Push your adapter and model card to the Hugging Face Hub and gather feedback.

© 2026 Tertiary Infotech Academy Pte Ltd

CONTINUE WITH TERTIARY INFOTECH

Recommended Courses

1. Generative AI with Large Language Models
2. Machine Learning with Python
3. Deep Learning with PyTorch
4. Retrieval Augmented Generation (RAG) with LangChain
5. Agentic AI Applications with Claude Code

© 2026 Tertiary Infotech Academy Pte Ltd

WE'RE HERE TO HELP

Support

1

Email
enquiry@tertiaryinfotech.com

2

Telephone
+65 6100 0613

3

Website
www.tertiarycourses.com.sg

4

LMS / TMS
https://lms-tms.tertiaryinfotech.com

5

Course page
www.tertiarycourses.com.sg/fine-tuning-open-source-llm.html

6

Venue
12 Woodlands Square #07-85/86/87, Singapore 737715

© 2026 Tertiary Infotech Academy Pte Ltd

Course Feedback

1

Share your feedback

Complete the course-feedback form before you leave today.

2

Trainer shares the link

The trainer displays the feedback form link and QR code.

3

Tell us what worked

Share what worked best and what needs more time.

4

Certificate of completion

Your e-certificate of completion is issued after the course.

© 2026 Tertiary Infotech Academy Pte Ltd

HAPPY FINE-TUNING

Thank You!

You can now build a fine-tuning dataset, adapt a pre-trained model with LoRA, QLoRA and Unsloth, and evaluate the result for quality, bias and benchmark alignment.

© 2026 Tertiary Infotech Academy Pte Ltd